

Actively Secure MPC in the Dishonest Majority Setting: Achieving Constant Complexity in Online Communication, Computation Per Gate, Rounds, and Private Input Size

Seunghwan Lee^{1,2}, Jaesang Noh¹, Taejeong Kim¹, Dohyuk Kim^{1,2}, and Dong-Joon Shin^{1,2}

¹ Department of Electronic Engineering, Hanyang University, Seoul, Korea
{kr3951, darkelzm, birdy0212, dohyuk1000, djshin}@hanyang.ac.kr

² waLLLnut Co., Ltd., Seoul, Korea
{shlee, dhkim, djshin}@walllnut.com

Abstract. SPDZ-style and BMR-style protocols are widely known as practical MPC protocols that achieve active security in the dishonest majority setting. However, to date, SPDZ-style protocols have not achieved constant rounds, and BMR-style protocols have struggled to achieve scalable communication or computation. Additionally, there exists fully homomorphic encryption (FHE)-based MPC protocols that achieve both constant rounds and scalable communication, but they face challenges in achieving active security in the dishonest majority setting and are considered impractical due to computational inefficiencies.

In this work, we propose an MPC framework that constructs an efficient and scalable FHE-based MPC protocol by integrating a linear secret sharing scheme (LSSS)-based MPC and FHE. The resulting FHE-based MPC protocol achieves active security in the dishonest majority setting and constant complexity in online communication, computation per gate, rounds, and private input size. Notably, when instantiated with the SPDZ protocol and gate FHE for the framework, the resulting FHE-based MPC protocol efficiently achieves active security in the dishonest majority setting by using SPDZ-style MAC and ensures the computation per gate time within 3 ms. Moreover, its offline phase achieves scalable communication and computation, both of which grow linearly with the number of parties n . In other words, the proposed FHE-based MPC preserves the key advantages of existing FHE-based MPCs and simultaneously overcomes the weaknesses of them. As a result, the proposed FHE-based MPC is a highly practical and secure like SPDZ-style and BMR-style protocols.

For the first time, we introduce the concept of circuit-private, which ensures that external adversaries who eavesdrop on communications do not obtain information about the circuit. We rigorously prove that our construction inherently satisfy circuit-privacy, thereby establishing a novel security option for MPC.

Keywords: Multiparty Computation, Fully Homomorphic Encryption, Dishonest Majority, Constant Complexity, Circuit Privacy

1 Introduction

A secure multiparty computation (MPC) protocol allows a set of n parties, $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$, to jointly compute a function f on their private inputs. During MPC protocol, a subset of parties $\text{Corrupt} \subseteq \mathcal{P}$ can be corrupted. Then, the security in an MPC protocol refers to the property that Corrupt gains no additional information about the private inputs of honest parties $\text{Honest} = \mathcal{P} \setminus \text{Corrupt}$ given only their inputs and outputs.

For many decades, MPC has principally been studied in cryptography from a theoretical perspective. In recent years, however, interest in its practical capabilities has grown substantially. Although MPC can be evaluated according to various attributes in a practical context, it is most commonly evaluated based on its communication complexity, computational complexity, and round complexity. In particular, both the communication and computational complexities are crucial for scalability, meaning the amortized work per party hopefully remains constant as the number of parties n grows [23]. Moreover, the round complexity is crucial for MPC to be independent of both circuit size $|\mathcal{C}|$ and the number of parties n . This paper focuses on the practical aspects of MPC protocols that achieve active security in the dishonest majority setting, which is recognized as the strongest security assumption in MPC. Such MPC protocols can be divided into SPDZ-style protocols, BMR-style protocols, and FHE-based MPC protocols.

The SPDZ protocol is a scalable MPC protocol that uses secret sharing and preprocessing model to achieve the communication complexity of $O(n|\mathcal{C}|)$ and the computational complexity per gate of $O(1)$ [25]. Several SPDZ-style protocols have been proposed to reduce the offline communication complexity, such as TopGear [4] optimizing ZKPoK (Zero-Knowledge Proof of Knowledge), and MASCOT [36] leveraging oblivious transfer (OT)-based MAC generation. Further improvements in the online phase include TurboSpeedz [7] halving the original online complexity, and SUPERPACK [30] reducing the original online complexity by up to a factor of $1/14$. Recent advancements in pseudo-random correlation generator scheme [9, 10, 52] have enabled SPDZ-style protocols [46] to achieve offline communication complexities of $O(n \log |\mathcal{C}|)$, along with $O(1)$ computational complexity per gate. However, the round complexity remains $O(|\mathcal{C}|)$, meaning the SPDZ-style protocols do not achieve the constant-round requirement.

The BMR protocol achieves a constant round in MPC by using a garbled circuit approach [5]. Recent BMR-style protocols have primarily focused on reducing both communication and computational complexities. Although early BMR protocol shows that the computational complexity per gate grows quadratically in the number of parties n , subsequent work [6] brought this complexity down to $O(1)$. Additionally, BMR-SPDZ protocol employs a preprocessing model to reduce online communication complexity [42]. While recent BMR-style protocols achieve constant-round MPC with $O(1)$ computation per gate, their online communication remains at $O(n^2|\mathcal{C}|)$ [35, 50]. Some BMR-style protocols manage

constant-round performance with $O(n|\mathcal{C}|)$ online communication, yet in practice, their computational complexity per gate grows with the number of parties n [34].

Although the development of SPDZ-style and BMR-style protocols indicates that simultaneously achieving constant round and scalable communication is challenging, it is possible to construct MPC protocols with both $O(1)$ communication and $O(1)$ rounds by employing fully homomorphic encryption (FHE) [26]. In particular, FHE-based MPC supports a single offline phase to generate evaluation key which can be used indefinitely unless the key is opened. However, as shown in Table 1, the existing FHE-based MPC protocols still face the following three practical issues compared to SPDZ-style and BMR-style protocols.

	Types	$ \text{ct} $	$ \text{ev} $	$ \text{evGen} $
GLS 15 [26]	ThHE	$O(nd_{\lambda,n}^2 \log d_{\lambda,n})$	0	0
JS23 [44]	ThHE	$O(d_{\lambda,n})$	$O(d_{\lambda,n}^2)$	$O(nd_{\lambda,n}^2)$
DDE+23 [22]	ThHE	$O(d_\lambda)$	$O(d_\lambda^2)$	unspecified
CCS19 [16]	MkHE	$O(nd_{\lambda,n})$	$O(n^2 d_{\lambda,n}^2)$	$O(n^2 d_{\lambda,n}^2)$
TLX+21 [54]	MkHE	$O(d_{\lambda,n})$	$O(d_{\lambda,n}^2)$	$O(nd_{\lambda,n}^2)$
Ours	ThHE	$O(d_\lambda)$	$O(d_\lambda^2)$	$O(n\kappa d_\lambda^2)$
	Majority	Security	Gate speed	NTT-friendly
GLS 15 [26]	Honest	Active	$O(\text{poly}(\lambda, n))$	Yes
JS23 [44]	Honest	Passive	$O(\text{poly}(\lambda, n))$	Yes
DDE+23 [22]	Honest	Active	$O(\text{poly}(\lambda))$	No
CCS19 [16]	Honest	Passive	$O(\text{poly}(\lambda, n))$	Yes
TLX+21 [54]	Honest	Passive	$O(\text{poly}(\lambda, n))$	Yes
Ours	Dishonest	Active	$O(\text{poly}(\lambda))$	Yes

Table 1: Comparison of FHE-based MPC protocols. The parameter λ is the computational security bit, κ is the statistical security bit, and $d_{\lambda,n}$ is the dimension of LWE sample (See Section 2.3), which is a function of λ and n (so does d_λ).

Absence of effective FHE-based MPC protocol with active security in the dishonest majority setting. Although the existing FHE-based MPC protocols can adopt ZKPoK to achieve active security in the dishonest majority setting, the use of ZKPoK for FHE make the protocol much slower and less efficient compared to SPDZ-style protocols [26]. Only recently have FHE-based MPC protocols effectively achieved active security exclusively in the honest majority setting [22].

Computational complexity per gate proportional to the number of parties n . As shown in Table 1, the computation per gate of the FHE-based MPC protocols except DDE+23 and Ours depends on both the number of parties n and the computational security parameter λ for LWE, whereas the computation per gate of DDE+23 and Ours depends only on λ . In other words, most FHE-based MPC protocols fail to achieve scalable computational complexity per gate.

Private input size per party $|\text{ct}| + |\text{ev}|$ proportional to the number of parties n . In the SPDZ-style protocols, the shares used as private input are generated using Shamir secret sharing, and hence each share size (modulus size) grows as the order of $\log n$. In contrast, FHE-based MPC utilizes a ciphertext ct and an evaluation key ev as private input, where $|\text{ct}|$ and $|\text{ev}|$ depend on LWE dimension $d_{\lambda,n}$ determined by both n and λ .

Among the three issues, 1 and 2 are the top considerations for future FHE-based MPC protocols suggested by NIST [1]. In recent works, DDE+23 proposes an MPC protocol achieving constant computational complexity per gate and addressing the issue 3. However, this protocol ensures active security only in the honest majority setting and is not NTT-friendly³ by requiring the modulus to be 2^k . Because modern FHEs commonly employ NTT with large moduli to effectively support repeated bootstrapping, being NTT-friendly is critical for making FHE-based MPC practical.

By examining the strengths and limitations of MPC protocols, we are naturally led to the following question.

Q. In the dishonest majority setting with active adversaries, does there exist a scalable MPC protocol that achieves constant complexity in online communication, computation per gate, round, and private input size?

Hence, in this paper, an FHE-based MPC protocol is proposed, which is scalable and actively secure in the dishonest majority setting to answer the aforementioned question. The contributions of this paper is listed next.

1.1 Contributions

We propose a novel framework for constructing FHE-based MPC protocols that integrate both linear secret sharing scheme (LSSS)-based MPC and FHE. As an example of this framework, the SPDZ protocol and homomorphic gate operational FHE are selected for LSSS-based MPC and FHE, respectively. Through this framework, the proposed FHE-based MPC protocol achieves the following three contributions, addressing the aforementioned issues and answering the proposed questions.

Active security in the dishonest majority setting using SPDZ-style MAC. For the first time, we propose an FHE-based MPC protocol that leverages the SPDZ-style MAC to efficiently achieve active security in the dishonest majority setting. Notably, the proposed framework for constructing FHE-based MPC supports all NTT-friendly FHE schemes, such as CKKS, BGV/FV, and FHEW, ensuring active security even in the dishonest majority setting.

Constant complexity in online communication, computation per gate, round, and private input size. The proposed FHE-based MPC achieves

³Here, NTT-friendly refers to whether a given MPC protocol can utilize NTT-based FHE schemes such as CKKS, FHEW, and BGV/FV, where the ciphertext modulus is formed as a product of primes.

constant complexity in online communication, computation per gate, rounds, and private input size regardless of both the number of parties n and circuit size $|\mathcal{C}|$ ⁴. In particular, the proposed MPC, which implements state-of-the-art gate FHE [39], achieves a gate operation speed of approximately 3 ms regardless of n . This contrasts with constant-round FHE-based MPC (330 ms for 4 parties and 900 ms for 16 parties [44]) and BMR-based MPC (0.5 ms for 2 parties and 3 ms for 32 parties in a LAN with 100 Mbps [34]), offering high performance and scalability. As shown in Table 1, private input size $|\text{ct}| + |\text{ev}|$ of the proposed FHE-based MPC protocol depends on the LWE dimension d_λ regardless of n .

Achieving scalable single offline phase and circuit privacy. Like other FHE-based MPC protocols, the proposed MPC protocol supports a single offline phase to generate evaluation key ev . In particular, the single offline phase is scalable because the communication and computation complexity in the offline phase is $O(n\kappa d_\lambda^2)$. In SPDZ-style and BMR-style protocols, a third party that does not participate in the protocol may eavesdrop and extract information about the circuit being computed by the parties. To address this issue, circuit privacy is introduced to ensure that no external adversary can obtain information about the circuit. Furthermore, it is shown that the proposed FHE-based MPC protocol achieves circuit privacy.

1.2 Technical Overview.

To achieve the aforementioned contributions, we present technical overview of our approach as follows.

1. Enhancing random bit sampling on a composite modulus from Rotaru et al. [48]. The key idea begins with generating SPDZ shares of random bits over a product-of-primes modulus $Q = p_1 \cdots p_l$. In particular, we improve upon the existing method of Rotaru et al. [48] in three main aspects:

- **Information leakage analysis.** Contrary to the claim in the existing method [48] that no information leakage occurs, we identify specific scenarios under which shares of the secret may leak information, and provide a full analysis of all possible cases.
- **Security without uniform randomness for corrupted parties.** While the existing method requires that the corrupted party’s share be sampled uniformly at random, our method remains secure even if the corrupted party arbitrarily chooses its share.
- **Reduced size of the auxiliary prime.** The existing method mandates the use of at least λ -bit auxiliary primes in the SPDZ protocol to achieve computational λ -bit security. In contrast, our approach removes the events that cause information leakage by employing rejection sampling. This enables κ -bit security with auxiliary prime p of fewer than 32 bits. Consequently, one can use machine-optimized primes p (with $|p| \leq 32$ bits) in $\mathbb{Z}_p$, improving computation speed compared to existing method using 128-bit primes.

⁴In this paper, the circuit size is defined solely by the number of gates.

2. Sharing the message from sampling a discrete distribution $\mathcal{D}_{\text{poly}(\lambda)}$ with support $[0, \text{poly}(\lambda))$ over a product-of-primes modulus. In LWE cryptosystems, discrete Gaussian noise is crucial distributions that enable provable security with the smallest variance. However, when designing FHE-based MPC, a corrupted party may attempt to use alternative samples as an attack strategy. To date, the only known method to guarantee security against such attacks is using a zero knowledge of proof of knowledge (ZKPoP), which relies on a knowledge proof that ensures the attacker’s samples remain within a bounded range [14], not the proof for the exact distribution knowledge.

Beyond secret sharing of random bits, we present a more general method to share samples from any discrete distribution with support bounded by $\text{poly}(\lambda)$ using an inverse transform sampling approach. For a statistical security parameter κ , our method requires at most $3\lceil \log \kappa \rceil + 3$ communication rounds and up to $7\text{poly}(\lambda)\kappa$ SPDZ multiplications, which means constant complexity in terms of the number of parties n . This result implies efficient sharing of discrete Gaussian samples in the presence of an active adversary is possible, with our framework.

3. Proposing secret-shared FHE and two new encryption/decryption methods. We introduce a cryptographic framework called *secret-shared FHE* (SSFHE). This framework is designed so that it can combine a variety of LSSS-based MPC or other secret-sharing schemes with different FHE schemes, subject to certain security requirements that we formally articulate. For a concrete analysis of our framework, we evaluate the round complexity and the total number of multiplications performed through SPDZ when executing FHE with GINX gate operations (referred to as GINX FHE [20, 39]). Even when using discrete Gaussian sampling, our method performs at most $O(d^2\kappa\sigma)$ SPDZ multiplications over $O(\log \kappa + \log \sigma)$ rounds in the LWE setting. Furthermore, if a binary secret key and binomial error distribution having constant variance are used, the number of SPDZ multiplication reduces to $O(d^2)$ within $O(1)$ rounds. Considering that the evaluation key size of GINX FHE is $O(d^2)$, our approach achieves optimal processing without incurring asymptotic overhead.

We also propose novel encryption and decryption algorithms that operate via LSSS-based MPC directly on the encryption and decryption circuits $\mathcal{C}_{\text{enc}}$ and $\mathcal{C}_{\text{dec}}$. Compared to the existing approaches using either heavy ZKPoP-based encryption or large integer moduli with smudging noise, our methods are significantly more efficient in practice.

4. Rigorous proof of UC-realizing circuit-private MPC. Finally, we define an ideal functionality for circuit-private MPC that ensures the evaluated circuit remains hidden in the presence of an eavesdropping adversary. This is particularly important in adversarial environments where multiple honest parties must collaborate without revealing which operations are being performed on their private data. Note that the existing research on appropriate FHE security definitions for FHE-based MPC has primarily relied on security proofs under additional assumptions. These include IND-CPAD [15, 17], which is defined by a security game where the adversary is given access to a decryption oracle for ciphertexts resulting from homomorphic operations, or circuit-private FHE [38],

which ensures the indistinguishability between a ciphertext obtained after FHE computation and a freshly encrypted ciphertext of the corresponding plaintext.

Surprisingly, unlike conventional LSSS-based MPC or garbled circuit-based MPC, we demonstrate that **UC-realization** of circuit-private MPC can be constructed solely based on the correctness and **IND-CPA** security of FHE, regardless of how the underlying FHE scheme is constructed. This is achieved because the LSSS-based MPC inherently masks all **LWE** ciphertexts intended for decryption, ensuring security without requiring additional assumptions.

1.3 Related work.

In this section, we introduce prior works that share similarities with the notion of circuit-private MPC as defined in this paper, as well as with the techniques used to achieve it.

1. Discrete Gaussian Sampling in the Shared State. In the field of differential privacy (DP), prior works have proposed adding noise to secret-shared data over a prime modulus to conceal sensitive information. In particular, techniques for sampling from the discrete Gaussian distribution have been studied [28, 51]. In contrast, this paper presents a method for approximating and sampling from a given distribution over secret shares with a composite modulus, which can be seen as a generalization of the approaches used in DP.

2. Private Circuit Evaluation via Universal Circuits. To hide the structure of the evaluation circuit—as in this work—prior research has considered the use of universal circuits, which take both the circuit description and its input as input and output the result of evaluation [53]. However, since the input length of a universal circuit grows with the size of the target circuit, this inherently leaks information about the circuit itself. In contrast, the definition of circuit-private MPC adopted in this paper only incurs leakage about the number of circuit inputs, thus providing a stronger notion of privacy.

2 Preliminaries

2.1 Notations

Given a row vector $\vec{a} = (a_1, \dots, a_d) \in \mathbb{Z}_p^d$, we define the negacyclic right rotation of $\vec{a}$ by i positions, denoted by $\text{rot}_i(\vec{a})$, as follows:

$$\text{rot}_i(\vec{a}) = (\underbrace{-a_{d-i+1}, -a_{d-i+2}, \dots, -a_d}_{i \text{ elements}}, \underbrace{a_1, a_2, \dots, a_{d-i}}_{d-i \text{ elements}}). \quad (1)$$

In addition, for a vector $\vec{a} \in \mathbb{Z}_p^d$, we define the square matrix $\text{ROT}(\vec{a})$ as

$$\text{ROT}(\vec{a}) = [\vec{a}^t \text{rot}_1(\vec{a})^t \text{rot}_2(\vec{a})^t \dots \text{rot}_{d-1}(\vec{a})^t] \in \mathbb{Z}_p^{d \times d}. \quad (2)$$

Definition 1 (Negligible). *For a function $f : \mathbb{N} \rightarrow \mathbb{R}$, we say f is negligible if for every positive integer c , there exists an integer N such that for all $n \geq N$,*

$$|f(n)| < n^{-c}.$$

Let negl denote the set of all negligible functions. We also use the abbreviation PPT to indicate *probabilistic polynomial time*.

A finite field $\mathbb{F}$ is a field satisfying $|\mathbb{F}| < \infty$, and the number of elements is always expressed as p^ϵ for some prime p and natural number ϵ . In this paper, we denote any prime field by $\mathbb{Z}_p$, where p is a prime number, and refer to it as the *base field*. When $\epsilon > 1$, we denote the finite field by either $\mathbb{F}_{p^\epsilon}$ (to specify its structure) or by $\mathbb{F}_q$, $q = p^\epsilon$, and refer to it as an *extension field*.

Additionally, let $\mathbb{Z}_{p_1}, \dots, \mathbb{Z}_{p_l}$ be base fields and let $Q = p_1 \cdot \dots \cdot p_l$. By the Chinese Remainder Theorem (CRT), there exist $\xi_1, \dots, \xi_l \in \mathbb{Z}_Q$ such that the following isomorphic map exists [21]:

$$\phi : \mathbb{Z}_{p_1} \times \dots \times \mathbb{Z}_{p_l} \rightarrow \mathbb{Z}_Q, \quad (x_1, \dots, x_l) \mapsto \sum_{i=1}^l \xi_i x_i. \quad (3)$$

Moreover, we have $\phi^{-1}(x) = (x \bmod p_1, \dots, x \bmod p_l)$. In this paper, we refer to the map ϕ in Eq. (3) as **GluingMap**.

Turing Machines and Protocols. In this paper, every PPT adversary $\mathcal{A}$ and every PPT distinguisher $\mathcal{D}$ are defined as a Turing Machine (TM) [49]. In particular, the distinguisher $\mathcal{D}$ refers to a TM whose output is a single bit.

Moreover, all parties $P_1, \dots, P_n$ in the set $\mathcal{P} \triangleq \{P_1, \dots, P_n\}$, the environment $\mathcal{Z}$ and the trusted third party TTP in this paper are defined as Interactive Turing Machines (ITMs) [33]. We call a program executed by a TM an *algorithm*, and a jointly executed program by ITMs a *protocol*. A protocol run by the parties $\mathcal{P}$ is denoted by Π , and the protocol for TTP is referred to as the ideal functionality $\mathcal{F}$.

We use the standard asymptotic notations $O(\cdot)$, $\Omega(\cdot)$, $\Theta(\cdot)$, and $o(\cdot)$. Furthermore, the number of parties is always denoted by n , the computational security parameter by λ , and the statistical security parameter by κ .

Circuit Complexity. In this paper, any circuit $\mathcal{C}$ is assumed to consist solely of additions and multiplications over a prime field $\mathbb{Z}_p$. We denote by $|\mathcal{C}|$ the number of multiplication gates in $\mathcal{C}$. The depth of the circuit, $\text{depth}(\mathcal{C})$, is defined with respect to its multiplication gates; that is, $\text{depth}(\mathcal{C})$ rounds suffice for computing all multiplication gates in $\mathcal{C}$.

Moreover, in this paper we denote the circuits $\mathcal{C}^{p_i}$ by using different primes p_i if the circuit is evaluated over $\mathbb{Z}_{p_i}$. For simplicity in analysis, we assume that the cost of every multiplication over $\mathbb{Z}_{p_i}$ is 1.

Linear Secret Sharing. Let q be an integer modulus, and let $x \in \mathbb{Z}_q$. We define the linear secret sharing $[x]^q \triangleq (x_1, \dots, x_n) \in \mathbb{Z}_q^n$ as uniformly sampling a decomposition $x = \sum_{i=1}^n x_i$ over $\mathbb{Z}_q$. In other words, among all ways to write x as a sum of n elements in $\mathbb{Z}_q$, we choose one uniformly at random. We then define $[x]_i^q \triangleq x_i$, which is the share for the party P_i , and we omit q and denoted $[x]^q$ as $[x]$ if the modulus is clear in the context.

2.2 Overview of Probability Theory

In this paper, a distribution $\mathcal{D} : \mathbb{Z} \rightarrow [0, 1]$ is always defined as a discrete probability mass function over $\mathbb{Z}$. Let $\text{supp}(\mathcal{D}) \subset \mathbb{Z}$ be the set of integers to which $\mathcal{D}$ assigns a nonzero probability, referred to as the *support* of $\mathcal{D}$. We may write $\mathcal{D}_B$ to indicate that $\text{supp}(\mathcal{D}) \subset [0, B]$.

Given a distribution $\mathcal{D}$, the cumulative distribution $\bar{\mathcal{D}}(x)$ is defined by $\bar{\mathcal{D}}(x) \triangleq \sum_{i=-\infty}^x \mathcal{D}(i)$. We use $X \stackrel{\$}{\leftarrow} \mathcal{D}$ to denote that X is a random variable (or sample) drawn according to the distribution $\mathcal{D}$.

Definition 2. Let $\mathcal{D}$ and $\mathcal{E}$ be two distributions, and let $X \stackrel{\$}{\leftarrow} \mathcal{D}$ and $Y \stackrel{\$}{\leftarrow} \mathcal{E}$ be their respective random variables (samples). The statistical distance $\Delta(\mathcal{D}, \mathcal{E})$ (equivalently $\Delta(X, Y)$) is defined as

$$\Delta(\mathcal{D}, \mathcal{E}) = \Delta(X, Y) \triangleq \frac{1}{2} \sum_{i \in \mathbb{Z}} |\mathcal{D}(i) - \mathcal{E}(i)|.$$

Furthermore, for any distinguisher $\mathfrak{D}$, the advantage $\text{adv}_{\mathfrak{D}}(X, Y)$ is defined as

$$\text{adv}_{\mathfrak{D}}(X, Y) \triangleq \left| \Pr[\mathfrak{D}(Z) = 1 \mid Z = X] - \Pr[\mathfrak{D}(Z) = 1 \mid Z = Y] \right|.$$

Definition 3. Let $\mathcal{D} = (\mathcal{D}^{(\lambda)})_{\lambda \in \mathbb{N}}$ and $\mathcal{E} = (\mathcal{E}^{(\lambda)})_{\lambda \in \mathbb{N}}$ be two (sequences) of distributions, and let $X^{(\lambda)} \stackrel{\$}{\leftarrow} \mathcal{D}^{(\lambda)}$ and $Y^{(\lambda)} \stackrel{\$}{\leftarrow} \mathcal{E}^{(\lambda)}$ be their respective random variables.

- We say that $\mathcal{D}$ and $\mathcal{E}$ are statistically indistinguishable if

$$\Delta(\mathcal{D}^{(\lambda)}, \mathcal{E}^{(\lambda)}) = \Delta(X^{(\lambda)}, Y^{(\lambda)}) \in \text{negl}(\lambda),$$

and we denote this relation by $\mathcal{D} \cong_s \mathcal{E}$.

- They are called computationally indistinguishable if for every PPT distinguisher $\mathfrak{D}$,

$$\text{adv}_{\mathfrak{D}}(X^{(\lambda)}, Y^{(\lambda)}) \in \text{negl}(\lambda),$$

and we denote this relation by $X \cong_c Y$.

In the above, we may omit the asymptotic parameter λ when it is clear from context. It is also known that both relations $\cong_s$ and $\cong_c$ are equivalence relations, and that $\mathcal{D} \cong_s \mathcal{E}$ implies $\mathcal{D} \cong_c \mathcal{E}$.

Useful Distributions. We define the uniform distribution $\mathcal{U}[a, b]$ as the distribution that assigns the same probability to every integer in the set $\{a, a+1, \dots, b\} \subset \mathbb{Z}$. We also denote $\mathcal{U}_q \triangleq \mathcal{U}[0, q-1]$.

Next, let $\mathcal{B}_k$ denote the binomial distribution obtained by sampling $u_1, \dots, u_k \stackrel{\$}{\leftarrow} \mathcal{U}_2$ and taking the integer sum $u_1 + \dots + u_k \in \mathbb{Z}$. We define $\mathcal{V}_{d,q}$ to be the distribution over $\mathbb{Z}_q^{d \times d}$ of the following two-step process: (i) Sample $\vec{a} \stackrel{\$}{\leftarrow} \mathcal{U}_q^d$; (ii) Compute $\text{ROT}(\vec{a}) \in \mathbb{Z}_q^{d \times d}$.⁵

⁵In this paper, the most crucial and consistently applied perspective and algebraic structure is the $\mathbb{Z}_q$ -vector space. For clarity, the Ring LWE and Module LWE defined over cyclotomic polynomial rings are also viewed in the form of such a structured lattice.

$\mathcal{DG}_\sigma$ denote the discrete Gaussian distribution with parameter σ defined by $\mathcal{DG}_\sigma(x) = \rho_\sigma(x) \left(\sum_{y \in \mathbb{Z}} \rho_\sigma(y) \right)^{-1}$, for $\rho_\sigma(x) = \exp(-x^2(2\sigma^2)^{-1})$. It is known that for $x \xleftarrow{\$} \mathcal{DG}_\sigma$, $\Pr[|x| > O(\sigma)] \leq \text{negl}$ [45].

Definition 4. Let $\mathcal{D} = (\mathcal{D}_\lambda)_{\lambda \in \mathbb{N}}$ be a distribution ensemble. We say that $\mathcal{D}$ is pseudorandom (denoted **pseudorandom**) if there exists a sequence of integer moduli $q = (q_\lambda)_{\lambda \in \mathbb{N}}$ such that

$$\mathcal{D} \cong_c \mathcal{U}_q.$$

We introduce an important lemma in this paper, which shows how to mask an arbitrary value b using a uniformly random a .

Lemma 1 (Masking Lemma). For any uniformly random $X \xleftarrow{\$} \mathcal{U}_B$ and any sample $Y \in \mathbb{Z}$ drawn from an arbitrary distribution (independently of X), the random variable $X + Y \bmod B$ is uniformly distributed and is independent of Y .

Proof. For a given $Y = j$, the function $f_Y(X) = Y + X \bmod B$ is a bijection. Hence, if X is uniformly random, $X + Y$ is also uniformly random. Moreover, for any $0 \leq i < B$, $\Pr[X + Y = i | Y = j] = \Pr[X = i - j] = B^{-1}$, which shows that X and Y are independent. $\square$

Inverse Transform Sampling. Suppose that a distribution $\mathcal{D}_B$ is given and that the probability values of its cumulative distribution $\bar{\mathcal{D}}_B$ can be represented with κ bits. Formally, for all $i \in \mathbb{Z}$, we have $2^\kappa \mathcal{D}_B(i) \in \mathbb{Z}$. Suppose further that we have a sample $x \xleftarrow{\$} \mathcal{D}_B$ and a sample $u \xleftarrow{\$} \mathcal{U}_{2^\kappa}$. Then the following relation holds [31, 41]:

$$\mathcal{D}_B(i) = \bar{\mathcal{D}}_B(i) - \bar{\mathcal{D}}_B(i-1) = \Pr[\text{argmin}_y \{u \leq 2^\kappa \bar{\mathcal{D}}_B(y)\} = i]. \quad (4)$$

Since $\bar{\mathcal{D}}_B(y)$ is nondecreasing, the value of argmin can be computed as follows:

$$z \leftarrow \text{argmin}_y \{u \leq 2^\kappa \bar{\mathcal{D}}_B(y)\} = \sum_{i=0}^B \text{compare}(2^\kappa \bar{\mathcal{D}}_B(i), u), \quad (5)$$

where $\text{compare}(a, b)$ outputs 0 if $a \leq b$ and 1 if $a > b$. In other words, by directly computing Eq. (5), we obtain a sample $z \xleftarrow{\$} \mathcal{D}_B$, and this method is known as *inverse transform sampling* (see [8]).

2.3 Overview of the Learning with Errors (LWE) Assumption and Fully Homomorphic Encryption (FHE)

This section provides an overview of the LWE assumption and fully homomorphic encryption (FHE).

Assumption 1 (LWE Assumption [43, 47]). Let $k, d, q, \mathbf{v} > 0$ be integer parameters, and let $\mathcal{D}_B$ be a B -bounded distribution⁶. Let $A \xleftarrow{\$} \mathcal{V}_{d,q}^{k \times \mathbf{v}}$. Then we assume

$$\vec{b} = \vec{s}A + \vec{e} \stackrel{\circ}{=} \vec{u} \in \mathbb{Z}_q^{d\mathbf{v}},$$

where $\vec{s} \xleftarrow{\$} \mathcal{D}_B^{dk}$, $\vec{u} \xleftarrow{\$} \mathcal{U}_q^{d\mathbf{v}}$, and $\vec{e} \xleftarrow{\$} \mathcal{D}_B^{d\mathbf{v}}$. We call this the LWE assumption.

We refer to $(A, \vec{b})$ as an LWE sample. In general, the error $\vec{e}$ in Assumption 1 is chosen from a discrete Gaussian $\mathcal{DG}$. However, for simplicity in this paper, we let $\vec{e}$ be sampled from $\{0, \dots, B-1\} \subset \mathbb{Z}_q$ with $B < q$.⁷ Note that distinguishing an LWE sample from a uniformly random sample is known to have a quantum reduction from the Short Independent Vectors Problem [47]. All cryptosystems appearing in this paper rely on the LWE assumption.

Definition 5 (Abstract FHE Cryptosystem [32]). An FHE cryptosystem Π_{FHE} is defined by four algorithms $(\text{KeyGen}^F, \text{Enc}_{\text{pk}}^F, \text{Dec}_{\text{sk}}^F, \text{Eval}_{\text{ev}}^F)$ (equivalently, four circuits $\mathcal{C}_{\text{KeyGen}}^F, \mathcal{C}_{\text{Enc}}^F, \mathcal{C}_{\text{Dec}}^F, \mathcal{C}_{\text{Eval}}^F$) as follows:

- KeyGen^F : Given the input 1^λ , choose integer parameters $k, d, q, \mathbf{v}, B > 0$ and a distribution $\mathcal{D}_B$. Sample $A \xleftarrow{\$} \mathcal{V}_{d,q}^{k \times \mathbf{v}}$, $\vec{s} \xleftarrow{\$} \mathcal{D}_B^{dk}$, $\vec{e} \xleftarrow{\$} \mathcal{D}_B^{d\mathbf{v}}$, and $\vec{r} \xleftarrow{\$} \mathcal{D}_B^{\text{poly}(\lambda)}$. Set $\text{sk} \leftarrow \vec{s}$. Then compute $(\text{pk}, \text{ev}) \leftarrow \mathcal{C}_{\text{KeyGen}}^F(\vec{s}, \vec{e}, \vec{r})$, as follows:
 1. Compute $\vec{b} = \vec{s}A + \vec{e}$, and set $\text{pk} = [A^t | \vec{b}^t] \in \mathbb{Z}_q^{dm \times (kd+d)}$.
 2. Derive ev from sk (see [11, 18, 20, 27] for details).
- Enc_{pk}^F : Given a message $\mathbf{m}$ and a (public) scaling factor Δ , sample $\vec{r} \xleftarrow{\$} \mathcal{D}_B^{d\mathbf{v}}$ and $\vec{e} = (e_1, \dots, e_d) \xleftarrow{\$} \mathcal{D}_B^d$. Then compute the ciphertext

$$\text{ct} \leftarrow \mathcal{C}_{\text{Enc}}^F(\Delta, \mathbf{m}, \vec{r}, \vec{e}) = \vec{r}\text{pk} + \underbrace{(0, \dots, 0)}_{kd}, \Delta\mathbf{m} + e_1, \dots, e_d \in \mathbb{Z}_q^{kd+d}.$$

- Dec_{sk}^F : Given $\text{ct} = (\vec{a}, \vec{b}) \in \mathbb{Z}_q^{kd+d}$ and the scaling factor Δ , compute $\mathcal{C}_{\text{Dec}}^F(\text{sk}, \text{ct})$ as follows:
 1. Compute $z \leftarrow b_1 - \vec{a}\text{sk}^t \in \mathbb{Z}_q$.
 2. Output $\mathbf{m}' \leftarrow \lfloor z\Delta^{-1} \rfloor$.
- (Abstract) $\text{Eval}_{\text{ev}}^F$: Given two ciphertexts ct_1 and ct_2 that encrypt messages $\mathbf{m}_1$ and $\mathbf{m}_2$, respectively, along with an specific homomorphic operation $*$ or a circuit $\mathcal{C}$, and ev , compute ct_{res} which encrypts $\mathbf{m}_1 * \mathbf{m}_2$. (See [11, 18, 20, 27]).

In this paper, we use KeyGen^F , Enc^F , Dec^F , and Eval^F to denote the algorithms of FHE cryptosystem, and every ciphertext ct is an FHE ciphertext. Depending

⁶In practice, one may choose different distributions for the secret key $\vec{s}$ and the error $\vec{e}$. However, for simplicity in analysis, we unify them into a single distribution $\mathcal{D}_B$.

⁷In Section 3.2, we will discuss how the discrete Gaussian distribution $\mathcal{DG}$ can also be viewed as a B -bounded distribution under our framework.

on how the evaluation key ev and the algorithm $\text{Eval}_{\text{ev}}^F$ are constructed, one can build different FHE schemes.

In Section 4.2, we employ an FHE scheme that supports GINX type gate homomorphic operations [20, 39] (denoted as GINX FHE). The circuit $\mathcal{C}_{\text{ev}}^F$ of GINX FHE is defined to produce multiple ciphertexts after obtaining an auxiliary pair $(\text{pk}', \text{sk}' = (s'_1, \dots, s'_{d'k'}))$. Concretely, it computes

$$\begin{aligned} & \text{Enc}_{\text{pk}}^F(s'_i, \Delta'_j) \quad \forall i \leq dk, j \leq l_1, \quad \text{and} \\ & \text{Enc}_{\text{pk}'}^F(\text{Equal}(s_i, b), \Delta''_j) \quad \forall i \leq d'k', b \leq B-1, j \leq l_2, \end{aligned} \quad (6)$$

where $l_1, l_2, \Delta', \Delta''$ are system parameters of GINX FHE, and $\text{Equal}(s_i, b)$ outputs 1 only if $s_i = b$, and 0 otherwise. We detail this process in Construction 2 of Section 4.2. In addition, when using Enc_{pk} in this paper, we omit the scaling factor Δ when it is clear from the context.

Definition 6 (IND-CPA Security and Correctness). *An FHE cryptosystem Π_{FHE} is δ_F -indistinguishable under chosen plaintext attack (δ_F -IND-CPA) if for any $l = O(\text{poly}(\lambda))$ messages $\{m_1, \dots, m_l\}$ and any PPT distinguisher $\mathcal{D}$,*

$$\text{adv}_{\mathcal{D}}\left((\text{pk}, \text{ev}, \text{Enc}_{\text{pk}}^F(m_1), \dots, \text{Enc}_{\text{pk}}^F(m_l)), (\text{pk}, \text{ev}, \text{Enc}_{\text{pk}}^F(m'_1), \dots, \text{Enc}_{\text{pk}}^F(m'_l))\right) \leq \delta_F.$$

An FHE cryptosystem is ϵ_F -correct if

$$\Pr\left[m_1 * m_2 \neq \text{Dec}_{\text{sk}}^F(\text{Eval}_{\text{ev}}^F(*, \text{ct}_1, \text{ct}_2)) \mid \text{ct}_i \leftarrow \text{Enc}_{\text{pk}}^F(m_i) \text{ for } i = 1 \text{ and } 2\right] \leq \epsilon_F$$

for any messages m_1, m_2 and any defined homomorphic operation $$. If both of these conditions hold, we call the cryptosystem (δ_F, ϵ_F) -FHE.*

If $\delta_F \in \text{negl}$ for a given cryptosystem $\Pi = (\Pi_\lambda)$, then we say that Π is IND-CPA secure. Moreover, under the LWE assumption, the sample $(\text{pk}, \text{Enc}_{\text{pk}}^F(m))$ for any message m is known to be pseudorandom [47].

Key-Dependent Message Assumption. Notice that after generating two public-secret key pairs (pk, sk) and (pk', sk') as in Eq. (6), each secret keys are encrypted by each other's public key. This construction is crucial for homomorphic encryption; in all known FHE schemes, one must encrypt a secret-key-dependent message (i.e., part of sk) to form the evaluation key ev , which is then made public in order to enable homomorphic operations. The following key-dependent message assumption ensures security despite revealing ev .

Definition 7 (Key-Dependent Message (KDM) Security [11]). *For $\Pi_{\text{FHE}} = (\text{KeyGen}^F, \text{Enc}_{\text{pk}}^F, \text{Dec}_{\text{sk}}^F, \text{Eval}_{\text{ev}}^F)$, we say Π_{FHE} has key-dependent message security if for any $l, k \in \text{poly}(\lambda)$ and any $\mathbb{Z}_q$ -valued functions $\Psi_1, \dots, \Psi_k$,*

$$\begin{aligned} & = \left(\text{pk}_1, \dots, \text{pk}_l, \text{Enc}_{\text{pk}_{j_1}}^F(\Psi_1(\text{sk}_{i_1})), \dots, \text{Enc}_{\text{pk}_{j_k}}^F(\Psi_k(\text{sk}_{i_k})) \right) \\ & \cong_c \left(\text{pk}_1, \dots, \text{pk}_l, \text{Enc}_{\text{pk}_{j_1}}^F(0), \dots, \text{Enc}_{\text{pk}_{j_k}}^F(0) \right), \end{aligned} \quad (7)$$

for any $1 \leq i_1, \dots, i_k, j_1, \dots, j_k \leq l$.

Assumption 2 (KDM Assumption). *We assume that Π_{FHE} satisfies key-dependent message security.*

Since all known FHE constructions rely on the KDM assumption, we also adopt Assumption 2 throughout this paper.

2.4 A Brief Introduction to the Universal Composability (UC) Framework

This paper assumes familiarity with the UC framework. Due to page limitations, we refer the reader to [13] or Supplementary Material A for further details. In this paper, we adopt the following assumption:

Assumption 3. *Every adversary $\mathcal{A}$ is an $(n - 1)$ -active and static PPT ITM. In other words, there is at least one honest party (full threshold), and $\mathcal{A}$ corrupts up to $n - 1$ parties before the protocol begins.*

We also assume that communication among parties takes place over a synchronized channel. Thus, all protocols proceed in an ordered fashion, and the resulting view is delivered to every party without omission, within a finite amount of time.

Ideal Functionality of $\mathcal{F}_{\text{CRS}}$

Rand: Upon receiving (Rand, i, q) from each party P_i , TTP forwards q to the adversary $\mathcal{A}$. Then TTP samples $u \xleftarrow{\$} \mathcal{U}_q$ uniformly at random in $\mathbb{Z}_q$. TTP sends u to all parties P_i and also to $\mathcal{A}$.

Fig. 1: The ideal functionality for sharing a common reference string (CRS).

Ideal Functionality $\mathcal{F}_{\text{CRS}}$. Note that, two distinct ideal functionalities can imply two distinct security notions when UC-realized. In Section 5, we propose a circuit-private MPC functionality $\mathcal{F}_{\text{CPMPC}}$. Compared to the arithmetic black-box (ABB)[36] functionality listed in Supplementary Material G, $\mathcal{F}_{\text{CPMPC}}$ provides additional privacy for the circuit $\mathcal{C}$ being evaluated.

2.5 Introduction to κ -Soundness LSSS-Based MPC and the SPDZ Protocol

This section introduces κ -soundness LSSS-based MPC and explains the SPDZ protocol [24, 25].

κ -Soundness LSSS-Based MPC. Let q be an integer modulus, and let $[\mathbf{m}_1]^q$ and $[\mathbf{m}_2]^q$ be linear secret sharings of $\mathbf{m}_1$ and $\mathbf{m}_2$. Then $[\mathbf{m}_1 + \mathbf{m}_2]^q$ can be computed locally by having each party P_i compute $[\mathbf{m}_1 + \mathbf{m}_2]_i^q \leftarrow [\mathbf{m}_1]_i^q + [\mathbf{m}_2]_i^q$. We say a protocol is a κ -soundness LSSS-based MPC if:

- (i) When generating shares for a message $\mathbf{m}$, the corrupted party P_j arbitrarily sets their share $[\mathbf{m}]_j^q$. The shares of the remaining honest parties, $[\mathbf{m}]_i^q$, are then chosen uniformly at random from the space satisfying $m = \sum_{i=1}^n [\mathbf{m}]_i^q$.
- (ii) There exists a protocol Π_{mul} that, given shares $[\mathbf{m}_1]^q$ and $[\mathbf{m}_2]^q$, outputs $[\mathbf{m}_1 \mathbf{m}_2]^q$. Let view_{mul} be the set of all communication values generated by the parties during the execution of Π_{mul} . Then, view_{mul} are independent to its messages $\mathbf{m}_1^q$ and $\mathbf{m}_2^q$, i.e., $\Pr[\mathbf{m}_1, \mathbf{m}_2] = \Pr[\mathbf{m}_1, \mathbf{m}_2 \mid \text{view}_{\text{mul}}]$.
- (iii) Given a circuit $\mathcal{C}$ with **poly** operations and shares of messages $[\mathbf{m}_1]^q, \dots, [\mathbf{m}_l]^q$, the probability that all parties fail to correctly reconstruct $\mathcal{C}(\mathbf{m}_1, \dots, \mathbf{m}_l)$ from the share $[\mathcal{C}(\mathbf{m}_1, \dots, \mathbf{m}_l)]^q$ via Π_{mul} is at most $2^{-\Theta(\kappa)}$ (in the presence of an **active** adversary).

Prior works have developed efficient methods for constructing Π_{mul} and robustly opening a share $[x]^q$, even under various types of adversaries $\mathcal{A}$, including those with **honest/dishonest majority**, γ -**passive/active**, **static/adaptive** corruption, and so forth [29]. In this paper, we focus on **SPDZ**, which remains secure and correct under an $(n-1)$ -**active** (dishonest majority) and **static** PPT adversary [24, 25].

SPDZ Share. To prevent the adversary $\mathcal{A}$ from breaking the correctness of the protocol, SPDZ uses an information-theoretic MAC key $\alpha \xleftarrow{\$} \mathbb{F}_{p^m}$. Each party P_i receives a share $[\alpha]_i$. The key α must remain unknown to all parties until the end of the protocol. Then, for each message $\mathbf{m} \in \mathbb{Z}_p$, the SPDZ share is defined as

$$\langle \mathbf{m} \rangle = ([\mathbf{m}]_1, \dots, [\mathbf{m}]_n, [\alpha \mathbf{m}]_1, \dots, [\alpha \mathbf{m}]_n, [\alpha]_1, \dots, [\alpha]_n), \quad (8)$$

where each local share for party P_i is $\langle \mathbf{m} \rangle_i = ([\mathbf{m}]_i, [\alpha \mathbf{m}]_i, [\alpha]_i)$. For a vector $\vec{\mathbf{m}}$, we similarly define $\langle \vec{\mathbf{m}} \rangle$ to be its SPDZ share.

Beaver Triple. In SPDZ, multiplication is performed using a Beaver triple $(\langle a \rangle^p, \langle b \rangle^p, \langle ab \rangle^p)$ for uniformly random $a, b \xleftarrow{\$} \mathcal{U}_p$. To evaluate a circuit $\mathcal{C}$ with $|\mathcal{C}|$ multiplication gates and $\text{depth}(\mathcal{C})$ levels, one needs $|\mathcal{C}|$ Beaver triples and $\text{depth}(\mathcal{C})$ communication rounds.

Due to page limitations, the detailed ideal functionality of the (Dynamic) SPDZ protocol is provided in Fig. 8 in Supplementary Material B. Throughout this paper, $\mathcal{F}_{\text{SPDZ}}^p$.Init takes a prime p as the message space, performs initialization, and is denoted as $\mathcal{F}_{\text{SPDZ}}^p$. Furthermore, we define the following protocols involving the TTP:

- The protocol where TTP generates a random share: $\mathcal{F}_{\text{SPDZ}}^p$.**Rand.**
- The protocol where TTP performs multiplication on two shares: $\mathcal{F}_{\text{SPDZ}}^p$.**Mul.**
- The protocol where TTP opens a share: $\mathcal{F}_{\text{SPDZ}}^p$.**Open.**
- The protocol where TTP verifies all opened shares: $\mathcal{F}_{\text{SPDZ}}^p$.**Check.**

Rachuri and Scholl [46] implement a protocol Π_{SPDZ} that realizes $\mathcal{F}_{\text{SPDZ}}$ in the $(\mathcal{F}_{\text{VOLE}}, \mathcal{F}_{\text{OLE}}, \mathcal{F}_{\text{CRS}})$ -hybrid model under the LPN assumption as summarized in Proposition 1.

Proposition 1 (Dynamic SPDZ [46]). *There exists a protocol Π_{SPDZ} that is an UC-realization of $\mathcal{F}_{\text{SPDZ}}$ in $(\mathcal{F}_{\text{VOLE}}, \mathcal{F}_{\text{OLE}}, \mathcal{F}_{\text{CRS}})$ -hybrid model under the LPN*

assumption. Moreover:

(i) If the MAC key lies in the extension field $\mathbb{F}_{p^e}$ and no **abort** occurs, the probability that any value opened via $\mathcal{F}_{\text{SPDZ}}.\mathbf{Open}$ differs from the correct result is $\Theta(p^{-e})$.

(ii) To generate $|\mathcal{C}|$ Beaver triples, the protocol needs at most $O(n \log |\mathcal{C}|)$ communication for seed exchange and at most $O(n \log |\mathcal{C}|)$ memory to store them. During the online phase, it requires at most $O(n|\mathcal{C}|)$ communication.

A central focus of this paper is to generate and securely open shares of $m \xleftarrow{\$} \mathcal{U}_2$ over the composite modulus $Q = p_1 \cdots p_l$, where each p_i is prime. For a single prime p , it is known (e.g., see Fig. 2) that one can produce a random bit share $[m]^p$ using a single multiplication [24, 25].

Protocol Π_{Bits}^p

Bits(i, m_i, p): Each party P_i calls $\mathcal{F}_{\text{SPDZ}}^p.\mathbf{Rand}$, $\mathcal{F}_{\text{SPDZ}}^p.\mathbf{Mul}$, and $\mathcal{F}_{\text{SPDZ}}^p.\mathbf{Open}$ in succession to obtain $\langle m \rangle^p$, $\langle m^2 \rangle^p$, and m^2 . Then it computes $v \leftarrow \sqrt{m^2}$, where v is either m or $-m$. Finally, it outputs $\langle (v^{-1} m + 1)/2 \rangle^p$.

Fig. 2: A protocol Π_{Bits}^p for obtaining share of a random bit.

Note that $\sqrt{m^2}$ is either m or $-m$, so the message in the resulting share is always a uniformly random bit. Thus, one can obtain a random bit share with only $O(1)$ multiplications.

In the next section, we propose CRT-SPDZ, in which the message space is extended to $Q = p_1 \cdots p_l$. We will show how to robustly generate a share of a sample $m \xleftarrow{\$} \mathcal{D}_B$ from a given distribution $\mathcal{D}_B$.

3 CRT-SPDZ and Its Application to Decryption, Indicator, and Inverse Transform Sampling Protocols

The main goal of this section is to define the CRT-SPDZ share over a product-of-primes modulus Q , and to propose and analyze two main protocols: (i) A protocol for decryption and indicator functionalities; (ii) A protocol that obtains a share of a sample $m \xleftarrow{\$} \mathcal{D}_{\text{poly}(\lambda)}$ from any distribution $\mathcal{D}_{\text{poly}(\lambda)}$. These protocols will be used in Section 4 for constructing a secret-shared FHE.

We begin by defining a new notion of secret sharing, $[[m]]^Q$, for a modulus Q that is a product of distinct (NTT-friendly) primes $p_1, \dots, p_l$.

Construction 1 (Protocol $\Pi_{\text{CRT-SPDZ}}$). *Let $p_1, \dots, p_l$ be distinct primes. Let κ be the statistical security parameter, and let $\epsilon_1, \dots, \epsilon_l$ be natural numbers such that $p_i^{-\epsilon_i} = 2^{-\Theta(\kappa)}$ for all i . Then the κ -soundness CRT-SPDZ protocol $\Pi_{\text{CRT-SPDZ}}$ is defined as follows:*

- **Secret Sharing** $[[\mathbf{m}]]^Q$: For a message $\mathbf{m} \in \mathbb{Z}_Q$, define $[[\mathbf{m}]]^Q \triangleq (\langle \mathbf{m} \bmod p_1 \rangle^{p_1}, \dots, \langle \mathbf{m} \bmod p_l \rangle^{p_l})$.
- **Init**: Each party interacts with the TTP to invoke $\mathcal{F}_{\text{SPDZ}}.\text{Init}$ for each prime p_j ($j = 1, \dots, l$). Let $\mathcal{F}_{\text{SPDZ}}^{p_j}$ denote the instance of $\mathcal{F}_{\text{SPDZ}}$ initialized with modulus p_j . Moreover, each party P_i obtains from TTP the collection of MAC keys $([\alpha_1]_i, \dots, [\alpha_l]_i)$, where $[\alpha_j]_i \in \mathbb{F}_{p_j}^{\epsilon_j}$. These keys collectively form the SPDZ-CRT MAC key for P_i , and are stored locally.
- **Input**: When a party P_i has an input $\mathbf{m}_i \in \mathbb{Z}_Q$, it computes $\mathbf{m}_i \bmod p_j \in \mathbb{Z}_{p_j}$ for each $j = 1, \dots, l$. Then P_i interacts with TTP via $\mathcal{F}_{\text{SPDZ}}^{p_j}.\text{Input}$ for each j , providing the modular reductions as inputs.
- **Mul, Open, and Check**: Suppose each party P_i holds two shares $[[\mathbf{m}_1]]^Q$ and $[[\mathbf{m}_2]]^Q$. Then to perform an multiplication between $\mathbf{m}_1$ and $\mathbf{m}_2$, the parties execute $\mathcal{F}_{\text{SPDZ}}^{p_j}.\text{Mul}$ with the $[[\mathbf{m}_1]]^Q$ and $[[\mathbf{m}_2]]^Q$ shares in each modulus p_j . Likewise, to open (or check) a share $[[\mathbf{m}_1]]^Q$, the parties execute $\mathcal{F}_{\text{SPDZ}}^{p_j}.\text{Open}$ (or, $\mathcal{F}_{\text{SPDZ}}^{p_j}.\text{Check}$) for each $\langle \mathbf{m}_1 \rangle^{p_j}$.

In essence, $\Pi_{\text{CRT-SPDZ}}$ applies SPDZ protocols in parallel for each of the l primes $p_1, \dots, p_l$, leveraging CRT to represent every $\mathbf{m} \in \mathbb{Z}_Q$ as $\mathbf{m}_1, \dots, \mathbf{m}_l$ with $\mathbf{m}_j \in \mathbb{Z}_{p_j}$. Moreover, the extension degrees ϵ_i for each $\mathbb{F}_{p_i}^{\epsilon_i}$ ensure $|\mathbb{F}_{p_i}^{\epsilon_i}| \approx 2^\kappa$. By Proposition 1 (adapted per prime p_i), if no **abort** occurs, then for any polynomial number of opened shares $[[\mathbf{m}_1]]^Q, \dots, [[\mathbf{m}_{\text{poly}(\lambda)}]]^Q$, the probability that an adversary $\mathcal{A}$ tampers with even one of these opened values is at most $O(2^{-\kappa})$. Finally, given the open values $\mathbf{m}_1, \dots, \mathbf{m}_l$ in $\mathbb{Z}_{p_1}, \dots, \mathbb{Z}_{p_l}$, one can recover the unique $\mathbf{m} \in \mathbb{Z}_Q$ via the **GluingMap** in Eq. (3).

Next, we introduce several key polynomials needed to implement decryption, indicator, and inverse transform sampling using only the $\Pi_{\text{CRT-SPDZ}}.\text{Mul}$ and $\Pi_{\text{CRT-SPDZ}}.\text{Open}$ protocols. We analyze the circuit size $|\mathcal{C}|$ (i.e., the number of multiplications or Beaver triples) and its multiplicative depth $\text{depth}(\mathcal{C})$. In $\mathcal{F}_{\text{SPDZ}}^p$, $|\mathcal{C}|$ corresponds to the number of $\mathcal{F}_{\text{SPDZ}}^p.\text{Mul}$ invocations, while $\text{depth}(\mathcal{C})$ corresponds to the number of communication rounds needed to evaluate $\mathcal{C}$.

Lemma 2 (Lagrange Interpolation [21]). *For any prime p and any function $f : I \subseteq \mathbb{Z}_p \rightarrow \mathbb{Z}_p$, there is a unique polynomial*

$$L(X) \triangleq \sum_{i \in I} f(i) \prod_{\substack{j \in I \\ j \neq i}} \frac{X - j}{i - j} \in \mathbb{Z}_p[X],$$

of degree at most $|I| - 1$, such that $L(\mathbf{m}) = f(\mathbf{m})$ for all $\mathbf{m} \in I$.

3.1 Decryption Polynomial $\Psi^p(X)$ and Indicator Polynomial $\psi_{B,j}^p(X) \in \mathbb{Z}_p[X]$

For simplicity, we first focus on a single prime p . Hence we use the notation $\langle x \rangle^p$.⁸ In Section 4.2, the proposed decryption protocol **Dec**^{new} must handle

⁸Since we separate the message m from the error e , one would take $p \geq 5$.

secret-shared values $\langle \lfloor p/2 \rfloor \mathbf{m} + e \rangle^p$, where $\mathbf{m} \in \{0, 1\}$ is a binary message and $0 \leq e < \lfloor p/2 \rfloor$. The polynomial is derived from Lagrange interpolation (Lemma 2) as follow:

$$\Psi^p(X) \triangleq \sum_{i=\lfloor p/2 \rfloor}^{p-1} \prod_{j=0, j \neq i}^{p-1} \frac{X-j}{i-j} = \sum_{i=\lfloor p/2 \rfloor}^{p-1} \prod_{j=0, j \neq i}^{p-1} (X-j) \in \mathbb{Z}_p[X]. \quad (9)$$

It is the unique polynomial of degree at most $p-1$ satisfying $\Psi^p(\lfloor p/2 \rfloor \mathbf{m} + e) = \mathbf{m}$ whenever $\mathbf{m} \in \{0, 1\}$ and $0 \leq e < \lfloor p/2 \rfloor$. Since $\prod_{j=0, j \neq i}^{p-1} (i-j) = (p-1)! \equiv 1 \pmod{p}$, the simplified product form on the right-hand side is derived.

Next, for any subset $\mathcal{I} \subset \mathbb{Z}_p$, and a message $\mathbf{m} \in \mathcal{I} \subset \mathbb{Z}_p$ the *indicator polynomial*

$$\Psi_{\mathcal{I},j}^p(X) \triangleq \prod_{i \in \mathcal{I}, i \neq j} \frac{X-i}{j-i} \in \mathbb{Z}_p[X]$$

uniquely outputs $\Psi_{\mathcal{I},j}^p(\mathbf{m}) = 1$ if $\mathbf{m} = j$ and $\Psi_{\mathcal{I},j}^p(\mathbf{m}) = 0$ otherwise, for any $j \in \mathcal{I}$. By Lemma 2, $\Psi_{\mathcal{I},j}^p(X)$ has degree at most $|\mathcal{I}|-1$. This polynomial corresponds to $\text{Equal}(\mathbf{m}, j)$ in Eq. (6), used in Section 4 for constructing FHE evaluation keys. We now analyze the circuit size and depth required to evaluate these polynomials.

Lemma 3. *There exist arithmetic circuits $\mathcal{C}_\Psi^p$ and $\mathcal{C}_{\Psi_{\mathcal{I},j}}^p$ such that for any $\mathbf{m} \in \mathbb{Z}_p$, $\mathcal{C}_\Psi^p(\mathbf{m}) = \Psi^p(\mathbf{m})$ and for any $\mathbf{m} \in \mathcal{I}$, $\mathcal{C}_{\Psi_{\mathcal{I},j}}^p(\mathbf{m}) = \Psi_{\mathcal{I},j}^p(\mathbf{m})$, which satisfy $|\mathcal{C}_\Psi^p| \leq p$, $|\mathcal{C}_{\Psi_{\mathcal{I},j}}^p| \leq |\mathcal{I}|$, $\text{depth}(\mathcal{C}_\Psi^p) \leq \lceil \log p \rceil$, and $\text{depth}(\mathcal{C}_{\Psi_{\mathcal{I},j}}^p) \leq \lceil \log |\mathcal{I}| \rceil$.*

The proof is provided in Supplementary Material E. We define the protocol Π_Ψ^p that invokes these circuits in Fig. 3.

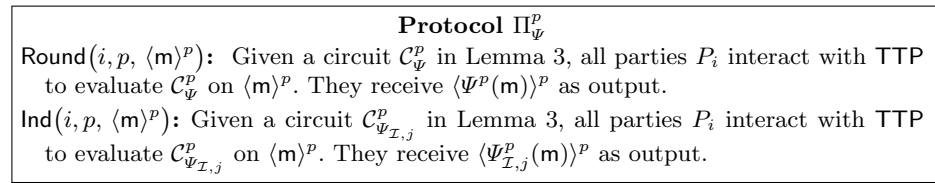


Fig. 3: A protocol Π_Ψ^p for rounding a shared value $\langle \mathbf{m} \rangle^p$ and indicating whether $\mathbf{m} = j$.

Remark. If one uses Π_Ψ^p .Round to convert the share $\langle \lfloor p/2 \rfloor \mathbf{m} + e \rangle^p$ into a share of the 1-bit message $\langle \mathbf{m} \rangle^p$, then no party must learn anything about the error e . This stands in contrast to prior methods that require large “smudging” errors to hide e [26, 44, 22, 16, 54]. In Section 5, we will show how to formally prove security without smudging errors in the UC-framework.

3.2 Inverse Transform Sampling for $\mathcal{D}_B$ Over a Prime Modulus p

In this section, we consider a prime modulus $Q = p$ and represent all secret shares in the form $\langle \mathbf{m} \rangle^p$. We introduce a technique to generate the share $\langle \mathbf{m} \rangle^p$ for samples $\mathbf{m} \xleftarrow{\$} \mathcal{D}_B$.

Before describing our main technique, we note that the proposed approach can generate $\langle \mathbf{m} \rangle^p$ for any distribution $\mathcal{D}$ admitting a $(B, \kappa, \epsilon, \mathcal{D})$ -distribution $\mathcal{D}_B^*$ (Definition 8). In particular, the existing protocols for FHE-based MPC generally cannot check whether each party truly uses an exact discrete Gaussian sample. By contrast, our protocol enables the secure sharing of samples from a discrete Gaussian $\mathcal{DG}_{\text{poly}(\lambda)}$, which allows minimal variance from a provable security standpoint.

Definition 8. For a distribution $\mathcal{D}$, $\mathcal{D}^*$ is called a $(B, \kappa, \epsilon, \mathcal{D})$ -distribution if the following three conditions hold:

1. $\text{supp}(\mathcal{D}^*) \subseteq \{0, 1, \dots, B-1\}$.
2. The cumulative distribution $\bar{\mathcal{D}}^*$ of $\mathcal{D}^*$ can be represented exactly with κ bits of precision.
3. There exists an integer constant C such that $\Delta(\mathcal{D}(\cdot), \mathcal{D}^*(\cdot - C)) \leq \epsilon$, where Δ denotes the statistical distance.

Remark. In the condition (3), C is a *translate factor* that allows us to approximate distribution whose support may include negative values. Sampling from $\mathcal{D}^*$ and then subtracting C yields a sample statistically indistinguishable from $\mathcal{D}$ if $\epsilon = \text{negl}$.

Proposition 2. For any distribution $\mathcal{D}$, precision κ , and bound B , there exists a $(B, \kappa, 1 - \Pr_{x \xleftarrow{\$} \mathcal{D}}[|x| < B/2] + (B+2)2^{-\kappa}, \mathcal{D})$ -distribution $\mathcal{D}^*$.

The proof is provided in Supplementary Material E.

Comparison Circuit $\mathcal{C}_{\text{comp}}^p$. Henceforth, we focus on a distribution $\mathcal{D}_B$ whose cumulative distribution $\bar{\mathcal{D}}_B$ can be represented with κ -bit precision. Our goal is to generate $\langle \mathbf{m} \rangle^p$ for $\mathbf{m} \xleftarrow{\$} \mathcal{D}_B$ using the *inverse transform sampling* method (Section 2.2). Concretely, inverse transform sampling requires computing $\text{compare}(x, y)$ many times, as in Eq. (5). Thus, if we build an arithmetic circuit $\mathcal{C}_{\text{comp}}^p$ over $\mathbb{Z}_p$ that implements $\text{compare}(\cdot, \cdot)$ using only additions and multiplications, we can share the inverse transform sample result.

Let $\wedge, \vee, \oplus$ denote the logical AND, OR, and XOR, respectively, and let $\neg$ denote negation. Suppose $\langle \mathbf{m}_1 \rangle^p$ and $\langle \mathbf{m}_2 \rangle^p$ are shares where each $\mathbf{m}_i \in \{0, 1\} \subset \mathbb{Z}_p$. Then we can implement:

$$\begin{aligned} \langle \mathbf{m}_1 \wedge \mathbf{m}_2 \rangle^p &= \langle \mathbf{m}_1 \mathbf{m}_2 \rangle^p, & \langle \mathbf{m}_1 \vee \mathbf{m}_2 \rangle^p &= 1 - \langle (1 - \mathbf{m}_1)(1 - \mathbf{m}_2) \rangle^p, \\ \langle \mathbf{m}_1 \oplus \mathbf{m}_2 \rangle^p &= \langle \mathbf{m}_1 + \mathbf{m}_2 - 2\mathbf{m}_1 \mathbf{m}_2 \rangle^p, & \langle \neg \mathbf{m}_1 \rangle^p &= 1 - \langle \mathbf{m}_1 \rangle^p. \end{aligned} \quad (10)$$

Observe that $\text{compare}(a, b)$ is determined by the sign of $(b - a)$. Let $a = \sum_{i=1}^{\kappa} a_i 2^{i-1}$, and $b = \sum_{i=1}^{\kappa} b_i 2^{i-1}$, where $\{a_i\}, \{b_i\} \subseteq \{0, 1\}^{\kappa}$. A known carry-lookahead design [12] computes

$$X_{\kappa} \vee \left(\bigvee_{i=1}^{\kappa-1} [X_i \wedge \left(\bigwedge_{j=i+1}^{\kappa} Y_j \right)] \right),$$

where $X_i = (\neg a_i) \wedge b_i$ and $Y_i = (\neg a_i) \oplus b_i$ for all $i = 2, \dots, \kappa$, $X_1 = a_1 \vee b_1$, and $Y_1 = a_1 \oplus b_1$. We implement these logical operations over $\mathbb{Z}_p$ using additions and multiplications as in Eq. (10). The following lemma bounds the circuit size and depth.

Lemma 4. *There exists circuit $\mathcal{C}_{\text{comp}}^p$ such that $|\mathcal{C}_{\text{comp}}^p| \leq 6\kappa$, $\text{depth}(\mathcal{C}_{\text{comp}}^p) \leq 3\lceil \log \kappa \rceil + 2$, and $\mathcal{C}_{\text{comp}}^p(\vec{a}, \vec{b}) = \text{Compare}(\sum_{i=1}^{\kappa} a_i 2^{i-1}, \sum_{i=1}^{\kappa} b_i 2^{i-1})$ for any two a, b with the binary vector representation $\vec{a} = (a_1, \dots, a_{\kappa}), \vec{b} = (b_1, \dots, b_{\kappa}) \in \{0, 1\}^{\kappa} \subset \mathbb{Z}_p$.*

The proof is provided in Supplementary Material E.

Inverse Transform Sampling on Shared Values. Let $\mathcal{D}_B$ be a distribution whose cumulative distribution $\bar{\mathcal{D}}_B$ is represented with κ bits, then κ bit representation of $\bar{\mathcal{D}}_B(i)$ is calculated as follows:

$$\vec{d}_i \triangleq (2^{\kappa} \bar{\mathcal{D}}_B(i) \bmod 2, 2^{\kappa-1} \bar{\mathcal{D}}_B(i) \bmod 2, \dots, 2 \bar{\mathcal{D}}_B(i) \bmod 2) \in \{0, 1\}^{\kappa}. \quad (11)$$

Note that after sharing random κ bits, performing the operation defined in Eq. (5) allows us to obtain a sample following $\mathcal{D}_B$. Consequently, we can generate a κ -bit uniformly random share $\langle u_1 \rangle^p, \dots, \langle u_{\kappa} \rangle^p$ and interpret above shares as a κ -bit integer. Then we can execute parallel **Compare** operations with the cumulative distribution in Eq. (11), and sum all the results, which is realization of the inverse transform sampling. We apply inverse transform sampling as in Fig. 4 to generate $\langle m \rangle^p$ for $m \xleftarrow{\$} \mathcal{D}_B$.

Protocol $\Pi_{\mathcal{D}}^p$

Sampling $(\mathcal{D}_B, B, \kappa, p, \{r_1^i, \dots, r_{B\kappa}^i\})$: Consider $\mathcal{C}_{\text{comp}}^p$ defined in Lemma 4.

- (i) Each party P_i runs protocol Π_{Bits}^p in Fig. 2 exactly κ times to obtain shares of uniformly random bits $\langle u_1 \rangle^p, \dots, \langle u_{\kappa} \rangle^p$.
- (ii) Let $\vec{u} \triangleq (u_1, \dots, u_{\kappa}) \in \{0, 1\}^{\kappa}$. In parallel, for each $j = 0, \dots, B-1$, the parties compute $\langle \text{compare}(\vec{u}, \vec{d}_j) \rangle^p$ using $\mathcal{C}_{\text{comp}}^p$ and shares $\langle u_i \rangle^p$, and $\vec{d}_j$ for $\bar{\mathcal{D}}_B(j)$ in Eq. (11).
- (iii) Set $\langle m \rangle^p \leftarrow \sum_{j=0}^{B-1} \langle \text{compare}(\vec{u}, \vec{d}_j) \rangle^p$, and output $\langle m \rangle^p$.

Fig. 4: A protocol $\Pi_{\mathcal{D}}^p$ for sampling from an $\mathcal{D}_B$.

We summarize the cost in terms of calls to $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ and the multiplicative depth as follows:

Lemma 5. *In the protocol $\Pi_{\mathcal{D}}^p$ in Fig. 4, the ideal functionality $\mathcal{F}_{\text{SPDZ.Mul}}$ is invoked at most $7B\kappa$ times within at most $3\lceil\log \kappa\rceil + 3$ communication rounds.*

The proof is provided in Supplementary Material E.

3.3 Sampling Protocol for $\mathcal{D}_B$ over a Composite Modulus Q

We now propose a method for sharing random bits in $[[\mathbf{m}]]^Q$ when $Q = p_1 p_2 \cdots p_l$ is a product of distinct primes p_i . We also give a protocol for sharing a sample from a distribution $\mathcal{D}_B$ defined over Q , where $B < \min_i p_i$.

Review of the Existing Method in [48]. Rotaru et al.[48] (denoted as the existing method) proposed a bit-sampling approach for a composite modulus Q , $Q = p_1 \cdots p_l$ with distinct primes p_i . By the CRT, each message bit $\mathbf{m} \in \{0, 1\} \subset \mathbb{Z}_Q$ corresponds to the tuple $(\mathbf{m}, \dots, \mathbf{m}) \in \prod_{i=1}^l \mathbb{Z}_{p_i}$. Hence, sampling a random bit $\mathbf{m}$ and creating $\langle \mathbf{m} \rangle^{p_1}, \dots, \langle \mathbf{m} \rangle^{p_l}$ yields $[[\mathbf{m}]]^Q$.

To generate t random bits concurrently, the existing method involves selecting an auxiliary prime p_0 , generating reference random bits using $\Pi_{\text{Bits}}^{p_0}$. Then, the resulting share $\langle \mathbf{m} \rangle^{p_0}$ satisfies $\mathbf{m} = \sum_{i=1}^n [\mathbf{m}]_i - \eta p_0 \in \mathbb{Z}$ for some integer η . If all parties obtain η , they can make $\langle [\mathbf{m}]_i^{p_0} \bmod p_k \rangle^{p_k}$ for $k = 1, \dots, l$, adjust by ηp_0 in each field, and thus assemble $[[\mathbf{m}]]^Q$.

Further, the existing method suggested using the value $\Delta = \lceil p_0/n \rceil$ and decomposing each share $[\mathbf{m}]_i^{p_0}$ as $l_i + \Delta h_i$ with $0 \leq l_i < \Delta$. Then, revealing the h_i values then allows parties to compute $\eta^* \leftarrow \lceil (\Delta \sum_i h_i) p_0^{-1} \rceil$. Finally, the existing method claimed: (i) No information on $\mathbf{m}$ is leaked; (ii) If each share $[\mathbf{m}]_i$ is uniformly random, then $\Pr[\eta \neq \eta^*] \leq 3p_0^{-1}$; (iii) For batching t bits at once, the scheme requires $p_0 > (t + \min_i \epsilon_i \lambda) 2^\lambda$, where ϵ_i is the extension degree of the MAC space $\mathbb{F}_{p_i^{\epsilon_i}}$, and λ is the computational security parameter. In addition, the existing method requires $\min_i \epsilon_i \lambda$ random bit shares to generate t random bit shares.

Analysis of the existing method. However, as an illustrative counterexample, consider a two-party scenario where the corrupted party P_2^* can recover $\mathbf{m}$ immediately after learning h_1

- **Toy example.** Let $p_0 = 7$, and let the corrupted share be $[\mathbf{m}]_2^7 = 1$. Then $\Delta = \lceil p_0/2 \rceil = 4$ and $h_2 = 0$. Because $\mathbf{m} = [\mathbf{m}]_1^7 + [\mathbf{m}]_2^7 \bmod 7$, we have $[\mathbf{m}]_1^7 \in \{0, 6\}$. If $h_1 = 0$, then $\mathbf{m} = 0$; otherwise $h_1 = 1$, then $\mathbf{m} = 1$. Therefore, the corrupted party thus learns $\mathbf{m}$ via h_1 .

Moreover, from the our proof of Proposition 3, we classify all cases where a corrupted party infers $\mathbf{m}$ if all l_i are 0, or all are $\Delta - 1$, or if $[\mathbf{m}]_i = p_0 - 1$. Thus $\mathbf{m}$ may leak, contrary to the existing method's claim (i).

The assumption in the existing method's (ii) that all corrupted shares $[\mathbf{m}]_i$ are also uniformly random may fail under standard SPDZ implementations, where the adversary can choose corrupted shares [24, 25]. Hence, the existing method is inapplicable.

Lastly let us consider the condition (iii). For the SSFHE proposed in Section 4 (based on GINX FHE[20, 39]), the chosen integer modulus often be ≤ 32 bits

or even ≤ 16 bits for high-speed arithmetic via modern computer architecture. Hence, requiring $p_0 \approx 2^{128}$ becomes impractical compared to other suitably chosen integer modulus.

Proposed Protocol Π_{Bits}^Q . Therefore, we simultaneously address the issues in the analysis of the existing method's (i), (ii), and (iii) above. Fig. 5 shows Π_{Bits}^Q , which defines an auxiliary prime p_0 and shares in $\mathcal{F}_{\text{SPDZ}}^{p_0}$. Based on our analysis, any sample that might leak information is discarded via rejection sampling, and by increasing the rejection rate through an integer parameter κ' , $p_0 > \max(2n(n+1), 2^{\kappa'} + t)$ can be reduced to a machine-friendly size $p_0 \approx 2^{32}$. Unlike prior methods based on the Multiple Subset-Sum Guessing problem, we can detect adversarial tampering via calling RepQ. Proposition 3 provides a full security and efficiency analysis.

Idea of the Proposed Protocol Π_{Bits}^Q . Based on our analysis, any sample causing information leakage is rejected (*rejection sampling*), addressing the analysis of the existing method's (i) and (ii). Moreover, by lowering κ' efficiently, the requirement $p_0 > \max(2n(n+1), 2^{\kappa'} + t)$ can be satisfied with $p_0 \approx 2^{32}$, resolving the analysis of the existing method's (iii). Rather than relying on the Multiple Subset-Sum Guessing assumption [48], we detect adversarial behavior via two calls to RepQ. Proposition 3 give a thorough security analysis of Π_{Bits}^Q in Fig. 5.

Proposition 3. *Let $t > 1$ be an integer, κ be a statistical parameter, κ' be a rejection parameter, and $p_0 > \max(2n(n+1), 2^{\kappa'} + t)$ be an auxiliary prime. Then:*

(i) *Let prob_1 denote the probability that Π_{Bits}^Q outputs abort in step (a) of Π_{Bits}^Q . Let assumed that no party is corrupted. Then, for $\alpha = 0$, $\text{prob}_1 \leq (t + \kappa\kappa')2n(n+1)p_0^{-1}$. Moreover, for sufficiently large $\alpha > 2n(n+1)(t + \kappa\kappa')(p_0 - 2n(n+1))^{-1}$,*

$$\log \text{prob}_1 \leq -(t + \kappa\kappa' + \alpha) \text{KL}\left(\alpha(t + \kappa\kappa' + \alpha)^{-1} \parallel 2n(n+1)p_0^{-1}\right),$$

where $\text{KL}(a||b) \triangleq a \log(ab^{-1}) + (1-a) \log((1-a)(1-b)^{-1})$ is the Kullback-Leibler divergence.

(ii) *Let prob_2 be the probability that Π_{Bits}^Q .RepQ returns Restart. Then $\text{prob}_2 \leq \kappa t 2^{-\kappa'}$.*

(iii) *If Π_{Bits}^Q .Bits outputs $[[\mathbf{m}_1]]^Q, \dots, [[\mathbf{m}_t]]^Q$, then for any $(n-1)$ -active and static adversary, each $\mathbf{m}_i$ is an independent uniformly random bit, except with probability at most $2^{-\Theta(\kappa)}$.*

The proof is in Supplementary Material E.

Concrete Parameter Selection. The proposed Π_{Bits}^Q protocol in Fig. 5 offers flexibility to choose parameters for various practical settings. For instance, let $\kappa \approx 60$, primes $p_1, \dots, p_t \approx 2^{16}$, and $p_0 \approx 2^{32}$ can be chosen such that Machine-level instructions handle these 16-bit or 32-bit primes and its arithmetic efficiently. If $\kappa' = 31$ and $t = 2^{10}$, one can set $\alpha = 23$ to ensure $\text{prob}_1 \approx 2^{-60}$ and $\text{prob}_2 \approx 2^{-15}$. Note that if a corrupted party maliciously attempts to force

Protocol Π_{Bits}^Q

The functionalities $\mathcal{F}_{\text{SPDZ}}^{p_0}, \mathcal{F}_{\text{SPDZ}}^{p_1}, \dots, \mathcal{F}_{\text{SPDZ}}^{p_l}$ are initialized.

BitsQ(t, α, κ'): Let $t' \leftarrow t + \kappa\kappa'$ and $t'' \leftarrow t' + \alpha$. Each party P_i calls $\Pi_{\text{Bits}}^{p_0}$. **Bits** t'' times to obtain $\langle u_1 \rangle^{p_0}, \dots, \langle u_{t''} \rangle^{p_0}$. Let $\Delta \leftarrow \lceil p_0/n \rceil$. Then, for all $\langle u_j \rangle^{p_0}$, the following each step is performed in parallel:

- (a) Each party P_j computes l_{ji}, h_{ji} such that $\langle u_j \rangle_i^{p_0} = l_{ji} + h_{ji}\Delta$ with $0 \leq l_{ji} < \Delta$, for all $j = 1, \dots, t''$. For each P_i , all indices j such that $l_{ji} = 0$, $l_{ji} = \Delta - 1$, or $\langle u_j \rangle_{p_0} = p_0 - 1$ are collected and broadcast. After receiving those index sets from all parties, take the union, exclude those shares, and then reorder to obtain $\langle u_1 \rangle^{p_0}, \dots, \langle u_{t'''} \rangle^{p_0}$. If $t''' < t'$, output **abort** and terminate.
 - (b) All parties P_i broadcast the values h_{ji} of the remaining shares, then compute $\eta_j \leftarrow \lceil (\Delta \sum_{i=1}^n h_{ji}) p_0^{-1} \rceil$.
 - (c) For each $k = 1, \dots, l$, each party P_i obtains $\langle u_{ji} \rangle^{p_k} \leftarrow \mathcal{F}_{\text{SPDZ}}^{p_k}.\text{Input}(\langle u_{ji} \rangle \bmod p_k)$.
 - (d) For each $k = 1, \dots, l$, compute $\langle m_j \rangle^{p_k} \leftarrow (\sum_{i=1}^l \langle u_{ji} \rangle^{p_k}) - (\eta_j p \bmod p_k)$.
- Then **RepQ**($t, [[m_1]]^Q, \dots, [[m_{t'}]]^Q$) is called. If **Restart** is returned, go back to the beginning; otherwise, return $[[m_1]]^Q, \dots, [[m_t]]^Q$.

RepQ($t, [[m_1]]^Q, \dots, [[m_{t'}]]^Q$): Among the shares, rename the $\kappa\kappa'$ shares $[[m_{t+1}]]^Q, \dots, [[m_{t+\kappa\kappa'}]]^Q$ as $[[u_{11}]]^Q, [[u_{12}]]^Q, \dots, [[u_{\kappa\kappa'}]]^Q$. All parties P_i perform the following each step in parallel for each $k = 0, 1, \dots, l$.

- (a) For $w = 1, \dots, \kappa$, after calling $\mathcal{F}_{\text{SPDZ}}$ to obtain random bits $r_{w1}, \dots, r_{wt}$, compute $\langle S_w \rangle^{p_k} \leftarrow \sum_{z=1}^t r_{wz} \langle m_z \rangle^{p_k} + \sum_{z=1}^{\kappa'} \langle u_{wz} \rangle^{p_k} 2^{z-1}$.
- (b) For $w = 1, \dots, \kappa$, open each $\langle S_w \rangle^{p_k}$ to obtain $S_w^{p_k}$.
- (c) For $w = 1, \dots, \kappa$, if $S_w^{p_0} < t$ or $S_w^{p_0} \geq 2^{\kappa'}$, output **Restart**.
- (d) For $k = 1, \dots, l$ and $w = 1, \dots, \kappa$, if $S_w^{p_0} \neq S_w^{p_k} \bmod p_k$ is found, output **abort** and terminate.

SamplingQ($t, \mathcal{D}_B$): Circuits $\mathcal{C}_{\text{comp}}^{p_1}, \dots, \mathcal{C}_{\text{comp}}^{p_l}$ satisfying Lemma 4 are given. Each party P_i calls Π_{Bits}^Q . **BitsQ**($t\kappa, \alpha, \kappa'$) and obtains the shares of random bit for t groups of κ each as follows. $[[u_{11}]]^Q, [[u_{21}]]^Q, \dots, [[u_{t\kappa}]]^Q$. Denote $\vec{u}_w \triangleq (u_{w1}, \dots, u_{w\kappa}) \in \{0, 1\}^\kappa$. Then, for all $\vec{d}_0, \dots, \vec{d}_{B-1}$ obtained from $\mathcal{D}_B$ in Eq. (11) and $w = 1, \dots, t$, the given circuit $\mathcal{C}_{\text{comp}}^p$ is executed in parallel B times. Compute $[[m_w]]^Q \leftarrow \sum_{j=0}^{B-1} [[\text{compare}(\vec{u}_w, \vec{d}_j)]]^Q$, and return $([[m_1]]^Q, \dots, [[m_t]]^Q)$.

Fig. 5: The protocol for sampling t random bits and a sample from $\mathcal{D}_B$ over the composite modulus $\mathbb{Z}_{Q=p_1 \dots p_l}$.

rejections, prob_1 remains statistically negligible and can serve as a detection mechanism. Meanwhile, prob_2 remains bounded by 2^{-15} under corruption if the protocol does not abort. Even if the protocol restarts, only auxiliary randomness leaks, allowing p_0 to be as small as under the 2^{32} .

Finally, in Proposition 4 we asymptotically analyze the number of **Mul** calls to $\mathcal{F}_{\text{SPDZ}}$ and the number of rounds in $\Pi_{\text{Bits}}^Q.\text{BitsQ}$ and $\Pi_{\text{Bits}}^Q.\text{SamplingQ}$ when p_0 is sufficiently large and $\alpha = 0$, respectively.

Proposition 4. *Let $Q = p_1 \cdots p_l$ be the product of l primes. Take $\alpha = 0$, $\kappa' = \kappa$, and $p_0 = 2^{\Theta(\kappa + \log n)}$. Then,*

- (i) *for generating $t = 2^{o(\kappa)}$ random bits, the protocol $\Pi_{\text{Bits}}^Q.\text{BitsQ}$ invokes $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ at most $t + \kappa^2$ times in a single communication round, and returns uniform bits except with probability $2^{-\Theta(\kappa)}$.*
- (ii) *for generate $t = 2^{o(\kappa)}$ samples from $\mathcal{D}_B$, the protocol $\Pi_{\text{Bits}}^Q.\text{SamplingQ}$ invokes $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ at most $7ltB\kappa + \kappa^2$ times within at most $3\lceil \log \kappa \rceil + 3$ communication rounds.*

The proof is in Supplementary Material E.

4 Secret-shared FHE Cryptosystem

In this section, we define the secret-shared FHE (SSFHE), in which the private key, public key, and evaluation key of an FHE scheme are secret-shared among the parties. We then propose a concrete SSFHE cryptosystem by integrating the $\Pi_{\text{CRT-SPDZ}}$ protocol from Section 3 with GINX FHE.

4.1 Definition of SSFHE and Analysis of Correctness and IND-CPA Security Reduction between (κ -soundness LSSS-based MPC, (ϵ_F, δ_F) -FHE) and (ϵ_S, δ_S) -SSFHE

The definition of SSFHE, denoted as the protocol Π_{SSFHE} , is given in Definition 9.

Definition 9 (SSFHE cryptosystem Π_{SSFHE}). *Let $[x]$ be a κ -soundness LSSS-based MPC, and let $(\text{KeyGen}^F, \text{Enc}_{\text{pk}}^F, \text{Dec}_{\text{sk}}^F, \text{Eval}_{\text{ev}}^F)$ be the FHE scheme with the implementation circuits $\mathcal{C}_{\text{KeyGen}}^F, \mathcal{C}_{\text{Enc}}^F, \mathcal{C}_{\text{Dec}}^F$, and $\mathcal{C}_{\text{Eval}}^F$, respectively. Then, the secret-shared FHE cryptosystem Π_{SSFHE} consists of three additional protocols $(\text{KeyGen}, \text{Enc}_{\text{pk}}, \text{Dec}_{[\text{sk}]^Q})$ defined as follows:*

- **KeyGen** (1^λ) : *Given input 1^λ , select a distribution $\mathcal{D}_B$ following the internal execution of $\text{KeyGen}^F(1^\lambda)$. Extract the modulus Q , which is the product of all integer moduli used in $\text{KeyGen}^F(1^\lambda)$. Generate shares $[r_1]^Q, \dots, [r_{\text{poly}(\lambda)}]^Q$ for randomly sampled values $r_1, \dots, r_{\text{poly}(\lambda)}$ drawn from $\mathcal{D}_B$. Using these shares as input, evaluate the circuit $\mathcal{C}_{\text{KeyGen}}^F$ to obtain $[\text{sk}]^Q, [\text{pk}]^Q$, and $[\text{ev}]^Q$. Open and reveal $[\text{pk}]^Q$ and $[\text{ev}]^Q$ to obtain pk and ev . Define $\text{view}_{\text{KeyGen}}$ as the collection of all communication values exchanged during the execution of KeyGen .*

- $\text{Enc}_{\text{pk}}(\mathbf{m})$: For a given message $\mathbf{m} \in \mathcal{M}$, generate the secret share $[\mathbf{m}]^Q$. Sample new random values $r_1, \dots, r_{\text{poly}(\lambda)}$ and generate their shares $[r_1]^Q, \dots, [r_{\text{poly}(\lambda)}]^Q$. Using $[\mathbf{m}]^Q$ and $[r_i]^Q$ as inputs, evaluate the circuit $\mathcal{C}_{\text{Enc}}^F$ to obtain the ciphertext $[\text{ct}]^Q$. Open $[\text{ct}]^Q$ to obtain the final ciphertext ct . Define view_{Enc} as the collection of all communication values exchanged during the execution of Enc_{pk} .
- $\text{Dec}_{[\text{sk}]^Q}(\text{ct})$: Generate shares $[r_1]^Q, \dots, [r_{\text{poly}(\lambda)}]^Q$ for newly sampled random values $r_1, \dots, r_{\text{poly}(\lambda)}$. Using ct , $[\text{sk}]^Q$, and $[r_i]^Q$ as inputs, evaluate the circuit $\mathcal{C}_{\text{Dec}}^F$ to obtain the message share $[\mathbf{m}]^Q$ corresponding to $\mathbf{m} \in \mathcal{M}$. Open $[\mathbf{m}]^Q$ to obtain the plaintext message $\mathbf{m}$. Define view_{Dec} as the collection of all communication values exchanged during the execution of $\text{Dec}_{[\text{sk}]^Q}$.

Finally, let $\text{view} \triangleq (\text{view}_{\text{KeyGen}}, \text{view}_{\text{Enc}}, \text{view}_{\text{Dec}})$ be the collection of all communication values generated in Π_{SSFHE} . SSFHE is ϵ_S -correctness if for any circuit $\mathcal{C}$ having the number of $\text{poly}(\lambda)$ gates,

$$\begin{aligned} \Pr[\mathcal{C}(m_1, \dots, m_l) \neq m' | \text{ct}_i \leftarrow \text{Enc}_{\text{pk}}(m_i) \text{ for all } 1 \leq i \leq l; \\ \text{ct}_{\text{res}} \leftarrow \text{Eval}_{\text{ev}}^F(\mathcal{C}, \text{ct}_1, \dots, \text{ct}_l); m' \leftarrow \text{Dec}_{[\text{sk}]^Q}(\text{ct}_{\text{res}}); \\ \text{Protocol does not abort}] \leq \epsilon. \end{aligned} \quad (12)$$

SSFHE is δ_S -IND-CPA secure if for any PPT distinguisher $\mathfrak{D}$, and any at most the $t - 1$ party's view, the following inequality holds:

$$\text{adv}_{\mathfrak{D}}\left((\text{pk}, \text{ev}, \text{ct}_{i_1}, \dots, \text{ct}_{i_l}, \text{view}), (\text{pk}, \text{ev}, \text{ct}_{j_1}, \dots, \text{ct}_{j_l}, \text{view})\right) \leq \delta, \quad (13)$$

for any message m_{i_k}, m_{j_k} in ambient message space $\mathcal{M}$ and its ciphertexts $\text{ct}_{i_k} \leftarrow \text{Enc}_{\text{pk}}(m_{i_k})$ and $\text{ct}_{j_k} \leftarrow \text{Enc}_{\text{pk}}(m_{j_k})$ are generated from m_{i_k} and m_{j_k} , respectively, where for each given index k , there exists a party $P_{k'}$ such that both ct_{i_k} and ct_{j_k} are generated by $P_{k'}$. When Π_{SSFHE} achieves ϵ -correctness and δ_S -IND-CPA, we call it as $(\epsilon_S, \delta_S) - \text{SSFHE}$.

Note that, unlike prior works that grant adversaries access to a specialized decryption oracle in IND-CPAD security models [15, 17], we only consider a weaker security notion of IND-CPA. Thus, compared to previous research, our definition imposes a lower security requirement. However, in Section 5.2, we will prove that (δ_S, ϵ_S) -SSFHE alone suffices for achieving circuit-private MPC via UC-realization.

Furthermore, as in Theorem 1, to the κ -soundness LSSS-based MPC and an (ϵ_F, δ_F) -FHE scheme, the correctness ϵ_S and security δ_S -IND-CPA of SSFHE can be related.

Theorem 1. *Given a κ -soundness LSSS-based MPC and an (δ_F, ϵ_F) -FHE scheme, it is possible to construct a $(\delta_S \leftarrow \delta_F, \epsilon_S \leftarrow 2^{-O(\kappa)} + \epsilon_F \text{poly}(\lambda))$ -SSFHE system.*

The proof is given in Supplementary Material E.

4.2 Proof of Existence: Protocols for Constructing SSFHE with $\Pi_{\text{CRT-SPDZ}}$ and FHE having CGGI-type Eval^F Algorithm

In this section, as a concrete example of SSFHE, we construct a protocol $\Pi_{\text{SSFHE}}^{\text{Bits}}$ in Construction 2, which combines the GINX FHE [20, 39]⁹ with the $\Pi_{\text{CRT-SPDZ}}$ protocol in Section 3, within the $(\mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{SPDZ}})$ -hybrid model. We then analyze its communication complexity. Furthermore, in Section 5, we define circuit-private MPC and prove that $\Pi_{\text{SSFHE}}^{\text{Bits}}$ in Construction 2 achieves UC-realization for circuit-private MPC.

Note that Construction 2 allows optional use of the known ZKPoP-based encryption Enc^{prev} [14] and smudging noise-based decryption Dec^{prev} [3]. Also, new algorithms Enc^{new} and Dec^{new} are proposed, allowing users to select the most suitable methods for their needs¹⁰.

First, the ZKPoP-based encryption is defined in the ideal functionality $\mathcal{F}_{\text{EncOracle}}$, as described in Fig. 6.

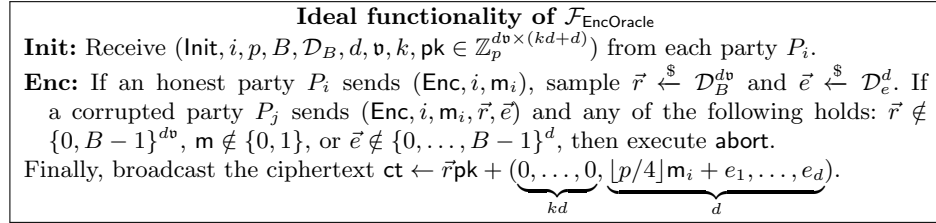


Fig. 6: The ideal functionality for the encryption oracle.

In order to construct a protocol that achieves UC-realization of $\mathcal{F}_{\text{EncOracle}}$ in Fig. 6, we rely on the fact that proving knowledge of a small $\vec{x}$ satisfying the relation $\vec{x}A = \vec{b} \in \mathbb{Z}_p^n$ can be reduced to implementing a ZKPoP protocol. Previous research [14] shows that lattice-based non-interactive ZKPoP protocols generate proofs faster than those utilizing SNARKs. However, a major drawback of such protocols is their slow proof generation and verification time, which can take several seconds. Moreover, they can only prove the boundedness of $\vec{x}$, rather than confirming that $\vec{x}$ is drawn from a specific distribution.

Next, we present Construction 2, which establishes the proof of existence for the proposed SSFHE in Section 4.1.

Construction 2 (SPDZ+GINX FHE). *The protocol $\Pi_{\text{SSFHE}}^{\text{Bit}}$ is defined as follows:*

– **Init**(1^λ) :

⁹Note that, other homomorphic encryption schemes such as CKKS [19] and BGV/FV [11] can be used to construct SSFHE within the framework proposed in this paper, provided that a κ -soundness LSSS-based MPC is available.

¹⁰It is also possible to mix different encryption and decryption methods, such as using Enc^{prev} with Dec^{new} , or Enc^{new} with Dec^{prev} .

- Given a computational security parameter 1^λ , select statistical parameter κ , distribution $\mathcal{D}_D$, prime moduli p_1 and p_{res} , composite moduli q_2 and q_3 such that q_2 and q_3 are product of primes, $p_{\text{res}} < p_1 < q_2 < q_3$ and $q_3 \gg 2^\kappa$.
- Select auxiliary prime modulus $p_0 = 2^{\Theta(\kappa + \log n)}$ and define $Q = p_1 q_2 q_3 p_{\text{res}}$. Let l be the number of all prime factors of Q such that $Q = p_1 \dots p_l$. Initialize $\mathcal{F}_{\text{GenSPDZ}}.\text{Init}(p_i)$ for all p_i and $\mathcal{F}_{\text{GenSPDZ}}.\text{Init}(p_0)$.
- Define FHE parameters $d_1, d_2, d_3, k_1, k_2, k_3, l_1, l_2, l_3, \sigma \in \mathbb{N}$, $N_1 = d_1 k_1, N_2 = d_2 k_2, N_3 = d_3 k_3$ and values $\Delta_{i_1 j_1}, \Delta'_{i_2 j_2}, \Delta''_{i_3 j_3}$ for all valid indices [20, 39].
- Set $f_{\text{com}}(\lambda) \triangleq l\sigma(N_1 + N_2 + N_3 + d_1 \mathbf{v} + l_1 d_1 N_2 + (l_2 d_2 + l_3 d_3) N_1) \in \text{poly}(\lambda)$.

– **KeyGen:**

- For all valid indices $i_1 \leq N_2, i_2, i_3 \leq N_1, j_1 \leq l_1, j_2 \leq l_2, j_3 \leq l_3$, and $-B \leq z \leq B$, run the following steps:
 1. Execute the protocol $\Pi_{\text{Bits}}^Q.\text{SamplingQ}$ polynomially many times and obtains shares $[[\vec{s}_1]]^Q, [[\vec{s}_2]]^Q, [[\vec{s}_3]]^Q, [[\vec{e}]]^Q, [[\vec{f}_{i_1 j_1}]]^Q, [[\vec{g}_{i_2 j_2}]]^Q$, and $[[\vec{h}_{i_3 j_3}]]^Q$ that satisfy $\vec{s}_1 \xleftarrow{\$} \mathcal{D}^{N_1}, \vec{s}_2 \xleftarrow{\$} \mathcal{D}^{N_2}, \vec{s}_3 \xleftarrow{\$} \mathcal{D}^{N_3}, \vec{e} \xleftarrow{\$} \mathcal{D}^{d_1 \mathbf{v}}, \vec{f}_{i_1 j_1} \xleftarrow{\$} \mathcal{D}^{d_1}, \vec{g}_{i_2 j_2} \xleftarrow{\$} \mathcal{D}^{d_2}, \vec{h}_{i_3 j_3} \xleftarrow{\$} \mathcal{D}^{d_3}$. Subtract a constant from all shares such that their expectation is zero.
 2. Run $\mathcal{F}_{\text{CRS}}$ to generate:

$$A_0 \xleftarrow{\$} \mathcal{V}_{d_1, p_1}^{k_1 \times \mathbf{v}}, \quad A_{1i_1 j_1} \xleftarrow{\$} \mathcal{V}_{d_1, p_1}^{k_1 \times 1}, \quad A_{2i_2 j_2} \xleftarrow{\$} \mathcal{V}_{d_2, q_2}^{k_2 \times 1}, \quad A_{3i_3 j_3} \xleftarrow{\$} \mathcal{V}_{d_3, q_3}^{k_3 \times 1},$$

for all $A_0 \in \mathbb{Z}_{p_1}^{d_1 k_1 \times d_1 \mathbf{v}}, A_{1i_1 j_1} \in \mathbb{Z}_{p_1}^{d_1 k_1 \times d_1}, A_{2i_2 j_2} \in \mathbb{Z}_{q_2}^{d_2 k_2 \times d_2}$, and $A_{3i_3 j_3} \in \mathbb{Z}_{q_3}^{d_3 k_3 \times d_3}$. We now define the following two functions:

$$\begin{aligned} \text{OneHot}_i^n : \mathbb{Z}_Q^n &\rightarrow \mathbb{Z}_Q^n, & (a_1, \dots, a_n) &\mapsto (a_i, 0, \dots, 0), \\ \text{IndVec}_{i,j}^n : \mathbb{Z}_Q^n &\rightarrow \mathbb{Z}_Q^n, & (a_1, \dots, a_n) &\mapsto (\text{Equal}(a_i, j), 0, \dots, 0). \end{aligned}$$

Calculate the following shares:

$$\begin{aligned} \langle \text{pk}_b \rangle^{p_1} &\leftarrow \langle \vec{s}_1 A_0 + \vec{e} \rangle^{p_1}, \\ \langle \text{ev}_{1i_1 j_1} \rangle^{p_1} &\leftarrow \langle \vec{s}_1 A_{1i_1 j_1} + \Delta_{j_1} \text{OneHot}_{i_1}^{d_1}(\vec{s}_2) + \vec{f}_{i_1 j_1} \rangle^{p_1}, \\ [[\text{ev}_{2i_2 j_2}]]^{q_2} &\leftarrow [[\vec{s}_2 A_{2i_2 j_2} + \Delta'_{j_2} \text{IndVec}_{i_2, z}^{d_2}(\vec{s}_1) + \vec{g}_{i_2 j_2}]]^{q_2}, \\ [[\text{ev}_{3i_3 j_3}]]^{q_3} &\leftarrow [[\vec{s}_3 A_{3i_3 j_3} + \Delta''_{j_3} \text{IndVec}_{i_3, z}^{d_3}(\vec{s}_1) + \vec{h}_{i_3 j_3}]]^{q_3}. \end{aligned}$$

3. Open all $\langle \text{pk}_b \rangle, \langle \text{ev}_{1i_1 j_1} \rangle^{p_1}, [[\text{ev}_{2i_2 j_2}]]^{q_2}$, and $[[\text{ev}_{3i_3 j_3}]]^{q_3}$, and set $[[\text{sk}_1]]^Q \triangleq [[\vec{s}_1]]^Q, \text{sk}_2 \triangleq [[\vec{s}_2]]^Q, \text{sk}_3 \triangleq [[\vec{s}_3]]^Q, \text{pk} \triangleq [A_0^t | \text{pk}_b^t]$, and $\text{ev} \triangleq \{A_{1i_1 j_1}, A_{2i_2 j_2}, A_{3i_3 j_3}, \text{ev}_{1i_1 j_1}, \text{ev}_{2i_2 j_2}, \text{ev}_{3i_3 j_3}\}$.

– **Enc^{new}(pk, m):**

- By using $\mathcal{F}_{\text{SPDZ}}^{p_1}$ run the following steps:
 1. Run $\mathcal{F}_{\text{SPDZ}}^{p_1}.\text{Input}$ to obtain $\langle \mathbf{m} \rangle^{p_1}$.
 2. Run $\Pi_{\mathcal{D}}^{p_1}.\text{Sampling}$ $d_1 \mathbf{v} + d_1$ times and obtain $\langle \vec{r} \rangle^{p_1}$ and $\langle \vec{e} \rangle^{p_1}$ for $\vec{r} \xleftarrow{\$} \mathcal{D}_B^{d_1 \mathbf{v}}$ and $\vec{e} \xleftarrow{\$} \mathcal{D}_B^{d_1}$.

3. Calculate

$$\langle \text{ct} \rangle^{p_1} \leftarrow \langle \text{rpk} \rangle^{p_1} + \langle \underbrace{(0, \dots, 0)}_{d_1 k_1}, \underbrace{\lfloor p_1/4 \rfloor \mathbf{m} + e_1, \dots, +e_{d_1}}_{d_1} \rangle^{p_1},$$

and open $\langle \text{ct} \rangle^{p_1}$ to obtain ct . Return ct .

- **Enc2^{prev}**($\text{pk}, \mathbf{m}$):
 - For a given input message $\mathbf{m}$ of the party P_i , run $\text{ct} \leftarrow \mathcal{F}_{\text{EncOracle}}.\text{Enc}(i, \mathbf{m})$. Return ct to all parties.
- **Dec^{new}**($[[\text{sk}]]^Q, \text{ct}$):
 - We assume that the message bits take values from $\{0, p_{\text{res}}/2\}^{11}$. If the modulus of ct is not equal to p_{res} , then return **abort** and terminate.
 - Otherwise let $\text{ct} = (\vec{a}, b) \in \mathbb{Z}_{p_{\text{res}}}^{k_1 d_1 + 1}$. Get $\langle \text{sk}_1 \rangle^{p_{\text{res}}}$ from $[[\text{sk}_1]]^Q$. Calculate $\langle \text{phase} \rangle^{p_{\text{res}}} \leftarrow \langle b - \text{sk}_1 \vec{a}^t \rangle^{p_{\text{res}}}$ locally.
 - Execute Π_{ψ}^p in Fig. 3 to obtain $\langle \mathbf{m} \rangle^{p_{\text{res}}} \leftarrow \langle \text{Round}_{p_{\text{res}}}(\text{phase}) \rangle^{p_{\text{res}}}$, Open $\langle \mathbf{m} \rangle^{p_{\text{res}}}$ and return $\mathbf{m}$.
- **Dec^{prev}**($[[\text{sk}]]^Q, \text{ct}$):
 - We assume that the message take values from $\{0, q_3/2\}^{11}$. If the modulus of ct is not equal to q_3 , then return **abort** and terminate.
 - Otherwise, execute the protocol Π_{Bits}^Q . **BitsQ** to obtain fresh κ shared bits $[[\mathbf{u}_1]]^Q, \dots, [[\mathbf{u}_{\kappa}]]^Q$. Get $[[\text{sk}_3]]^{q_3}$ and $[[\mathbf{u}_1]]^{q_3}, \dots, [[\mathbf{u}_{\kappa}]]^{q_3}$ from $[[\text{sk}_3]]^Q, [[\mathbf{u}_1]]^Q, \dots, [[\mathbf{u}_{\kappa}]]^Q$, respectively.
 - Let $\text{ct} = (\vec{a}, b) \in \mathbb{Z}_{q_3}^{k_3 d_3 + 1}$ and let $[[E]]^{q_3} \leftarrow \sum_{i=1}^{\kappa} 2^{i-1} [[\mathbf{u}_i]]^{q_3}$. Calculate $[[\text{phase}]]^{q_3} \leftarrow [[b - \text{sk}_3 \vec{a}^t + E]]^{q_3}$ locally.
 - Open $[[\text{phase}]]^{q_3}$ and obtain $\text{phase} \in \mathbb{Z}_{q_3}$. Return $\mathbf{m} \leftarrow \lceil \text{phase}(2/q_3) \rceil$.

Rationale for Choosing the Moduli p_1, q_2, q_3 , and p_{res} . Due to page limitations, the existing criteria for selecting p_1, q_2, q_3 , and p_{res} are discussed in detail in Supplementary Material C.

Now, we analyze the advantages and disadvantages of the proposed encryption and decryption protocols compared to the existing methods.

Comparison between **Enc^{prev} and **Enc^{new}**.** First, the existing **Enc^{prev}** in Construction 2 has been implemented using non-interactive ZKPoP-based encryption $\mathcal{F}_{\text{EncOracle}}$ [14]. In this case, a party P_i can execute **Enc^{prev}** in just on communication rounds by broadcasting its ciphertext and proof to all parties. Additionally, **Enc^{prev}** can be used even by a new party P^* that did not participate in the initial KeyGen. However, a significant drawback is that proof generation takes several seconds.

In contrast, the proposed **Enc^{new}** allows pre-generation and storage of all shares such as $\langle \vec{r} \rangle$ and $\langle \vec{e} \rangle$, except for the secret share of the message $\mathbf{m}$. Therefore, once the secret share of $\mathbf{m}$ is generated, encryption can be performed quickly using a precomputed random share-opening protocol [24, 25]. Furthermore, since

¹¹ The homomorphic operation algorithm Eval^F of a given FHE can always be performed with an arbitrarily defined message space. Therefore, if the ciphertext modulus used for decryption is q , we assume that in the preceding computations, the most significant bit has always been set as the message bit, i.e., $\{0, q/2\}$.

verification requires only MAC checking, the proof generation is extremely fast. If Dec^{new} is used, a single proof suffices for both generation and verification. However, a drawback is that it requires at least 2 rounds of communication, meaning the protocol may be delayed in environments with poor network conditions.

Comparison between Dec^{prev} and Dec^{new} . The existing Dec^{prev} in Construction 2 requires only a single share opening which significantly reduces the number of communication rounds. However, to transform the ciphertext modulus into a sufficiently large $q_3 \gg 2^\kappa$, a costly bootstrapping algorithm must be used, resulting in a slow decryption time ranging from several hundred milliseconds to a few seconds [22].

In contrast, the proposed Dec^{new} utilizes a much smaller modulus $p_{\text{res}} \approx 2^{10}$, which allows decryption to be implemented using at most $d_1 k_1 + 1$ modular reduction operations, resulting in the execution time in the nanosecond range. Subsequently, up to p_{res} multiplications are performed over $\lceil \log p_{\text{res}} \rceil$ rounds. However, since modern computing hardware can execute operations over $\mathbb{Z}_{p_{\text{res}}}$ in tens of nanoseconds, the total execution time, including multiplications and additions, is in the tens of microseconds. Thus, in terms of computational efficiency, Dec^{new} achieves an over three orders-of-magnitude speedup. However, a potential drawback is the additional $\lceil \log p_{\text{res}} \rceil$ communication rounds, which may cause delays in networks with poor connectivity.

4.3 Analyzing the Number of Calls to $\mathcal{F}_{\text{SPDZ.Mul}}$ and Communication Rounds in $\Pi_{\text{SSFHE}}^{\text{Bit}}.\text{KeyGen}$

In the proposed protocol $\Pi_{\text{SSFHE}}^{\text{Bit}}$, the KeyGen requires much more calls to $\mathcal{F}_{\text{SPDZ.Mul}}$ compared to Enc^{new} and Dec^{new} . Thus, in this section, we analyze the maximum number of calls to $\mathcal{F}_{\text{SPDZ.Mul}}$ and the maximum number of communication rounds required during KeyGen process in Theorem 2. Note that once KeyGen is executed, it does not need to be re-executed unless the set of participating parties changes.

Theorem 2. *Suppose we execute $\Pi_{\text{SSFHE}}^{\text{Bit}}.\text{KeyGen}$ in Construction 2.*

- *If $\mathcal{D}_B$ is chosen as $\mathcal{DG}_\sigma$, this protocol requires at most $O(\log \sigma + \log \sigma)$ communication rounds and at most $O(f_{\text{com}}(\lambda)\kappa + \kappa^2 + lN_1\sigma^2)$ calls to $\mathcal{F}_{\text{SPDZ.Mul}}$.*
- *If $\mathcal{D}_B$ is chosen as $\mathcal{B}_\sigma$, the protocol requires at most $O(\log \sigma)$ communication rounds and at most $O(f_{\text{com}}(\lambda)\kappa + \kappa^2 + lN_1\sigma^2)$ calls to $\mathcal{F}_{\text{SPDZ.Mul}}$.*

The proof is provided in Supplementary Material E.

Concrete Analysis of the Number of Calls to $\mathcal{F}_{\text{SPDZ.Mul}}$ while Running $\Pi_{\text{SSFHE}}^{\text{Bit}}.\text{KeyGen}$. The function $f_{\text{com}}(\lambda)$ consists of various parameters, making it difficult to intuitively determine the complexity. In this paper, we discuss how $f_{\text{com}}(\lambda) \approx O(d^2)$ can be achieved in practical scenarios. As a result, in the case of a discrete Gaussian, $\mathcal{F}_{\text{SPDZ.Mul}}$ is invoked $O(d^2\kappa)$ times over $O(\log \kappa)$ communication rounds, whereas for a binomial distribution, it is invoked $O(d^2)$ times over $O(1)$ communication round. Due to page limitations, further details are provided in Supplementary Material D.

In the next section, we discuss how the proposed protocol $\Pi_{\text{SSFHE}}^{\text{Bit}}$ in Construction 2 provides a new advantage of circuit privacy, a feature that has not yet been achieved in LSSS-based MPC or garbled circuit-based MPC.

5 Circuit-private MPC and its UC-realization

In this section, we compare the functionality of traditional MPC with the proposed circuit-private MPC functionality, demonstrating that the latter provides a stronger security definition. We then show that the proposed $\Pi_{\text{SSFHE}}^{\text{Bit}}$ achieves the UC-realization of circuit-private MPC.

5.1 Previous Definition of Ideal Functionality $\mathcal{F}_{\text{ABB}}$ and its Limitations

Traditional MPC protocols, such as the arithmetic black-box model $\mathcal{F}_{\text{ABB}}$ depicted in Fig. 9 of Supplementary Material G or the SPDZ functionality $\mathcal{F}_{\text{SPDZ}}$ in Section 3, operate by transmitting private inputs $\mathbf{m} \in \mathbb{F}_q$ to a TTP. The protocol is designed to securely invoke the **Mult** and **Add** operations so that TTP can perform computations between variables stored in its registers. Finally, parties request the output of specific registers to obtain the result of a given arithmetic circuit $\mathcal{C}$.

However, while these definitions guarantee the privacy of parties' inputs, they reveal information about the circuit structure to any entity that observes the protocol execution. Specifically, an external party can infer that **Add** and **Mult** operations are being performed, thereby exposing circuit-related information. For instance, in LSSS-based MPC, the amount of communication is proportional to the number of multiplications, and the number of communication rounds depends on the circuit depth, thus leaking key details about the circuit structure. Similarly, in garbled circuit-based MPC, the number of wires and the circuit size are directly reflected in the communication overhead.

Moreover, if a given MPC protocol is used repeatedly for a fixed set of circuits $\mathcal{C}_1, \dots, \mathcal{C}_l$, then the information revealed in communication can serve as a fingerprint, enabling parties to infer which circuit is being executed.

5.2 Circuit-private Ideal Functionality $\mathcal{F}_{\text{CPMPC}}$

In this section, we define circuit-private MPC through the ideal functionality $\mathcal{F}_{\text{CPMPC}}$, as depicted in Fig. 7, and explain how it ensures the privacy of circuit $\mathcal{C}$ from parties that do not participate in the protocol.

In Fig. 7, the ideal functionality $\mathcal{F}_{\text{CPMPC}}$ maintains a **State** register that records previously issued commands, enforcing a strict sequential execution of **CircuitInit**, **Input**, and **Output**. Additionally, $\mathcal{F}_{\text{CPMPC}}$ directly receives the circuit $\mathcal{C}$ from all parties and discloses only the number of inputs and the mapping of inputs to parties to the adversary $\mathcal{A}$.

Ideal functionality of $\mathcal{F}_{\text{CPMPC}}$	
Init:	On input (init, t) from all parties, store the threshold t . Initialize the register State to 0.
CircuitInit:	On input $(\text{CircInit}, i, \mathcal{C}, l)$ from all parties P_i , if State is 0, store the circuit $\mathcal{C}$ and leak the number of inputs l and the input-to-party mapping to the adversary $\mathcal{A}$. Set State to 1.
Input:	On input $(\text{input}, P_i, \text{varid}, x)$ from all parties P_i and $(\text{input}, P_i, \text{varid}, ?)$ from all other parties in $\mathcal{P} \setminus P_i$, store (varid, x) if varid is a fresh identifier. Set State to 2.
Output:	On input $(\text{output}, i, \text{varid})$ from all parties P_i , if varid exists in memory, TTP retrieves (varid, x) and outputs it to the adversary $\mathcal{A}$. If $\mathcal{A}$ inputs OK, x is output to all parties, and State is reset to 0. If $\mathcal{A}$ inputs $\perp$, then $\perp$ is output to all parties, and the protocol terminates.

Fig. 7: The ideal functionality for circuit-private MPC.

If no corrupted party exists (equivalently, if the adversary $\mathcal{A}$ is an external entity that does not participate in the MPC protocol), we analyze the key differences between $\mathcal{F}_{\text{CPMPC}}$ and the traditional MPC functionalities as follows. Since $\mathcal{F}_{\text{CPMPC}}$ does not contain explicit **Add** or **Mult** commands, but instead directly receives and evaluates $\mathcal{C}$, the circuit $\mathcal{C}$ remains a private input for all parties from the adversary's perspective. Thus, if no corrupted party exists, the only information that $\mathcal{A}$ can learn about $\mathcal{C}$ is its input size and the mapping between inputs and parties.

If at least one corrupted party exists, the adversary $\mathcal{A}$ can obtain information about $\mathcal{C}$ through the corrupted party. In this case, the adversary's knowledge about $\mathcal{C}$ becomes equivalent to that in a standard MPC setting. Hence, the key distinction between traditional MPC and circuit-private MPC arises in scenarios where no corrupted party is present.

For LSSS-based MPC and garbled circuit-based MPC, adversaries who lack prior knowledge of $\mathcal{C}$ face significant challenges in simulating the number of **view** instances produced by the protocol. In contrast, in the case of $\Pi_{\text{SSFHE}}^{\text{Bit}}$ from Construction 2, all instances of **view** follow a distribution that is independent of the message. Thus, as stated in Theorem 3, we establish that $\Pi_{\text{SSFHE}}^{\text{Bit}}$ achieves the UC-realization of $\mathcal{F}_{\text{CPMPC}}$.

The key idea behind the proof of Theorem 3 is that the environment $\mathcal{Z}$ does not obtain the resulting ciphertext ct_{res} after computation. If Dec^{prev} is used, the only observable values are the sum of the output message and the smudging error, i.e., $m + E$, and the independently generated shares $[m + E]_1, \dots, [m + E]_n$. Since these values can be easily simulated, the environment cannot distinguish between real and ideal executions. Note that the two protocols, **EXEC** and **IDEAL**, are described in detail in Supplementary Material A.

Theorem 3. *Consider Assumptions 1, 2, and 3. Let $\Pi_{\text{SSFHE}}^{\text{Bit}}$ be an (ϵ_S, δ_S) -SSFHE scheme with κ -soundness SPDZ. Assume that all errors in **ct** before performing Dec^{prev} or Dec^{new} are bounded by B . Then, for any environment $\mathcal{Z}$ and adversary $\mathcal{A}$, there exists an adversary $\mathcal{A}^*$ such that*

$$\text{adv}_{\mathcal{Z}}(\text{EXEC}_{\Pi_{\text{SSFHE}}^{\text{Bit}}, \mathcal{A}, \mathcal{Z}}, \text{IDEAL}_{\mathcal{F}_{\text{CPMPC}}, \mathcal{A}^*, \mathcal{Z}}) \leq \delta_S + (B - 1)2^{-\kappa+2} + 2\epsilon_S,$$

under the $(\mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{SPDZ}})$ -hybrid model.

The proof is provided in Supplementary Material F. Thus, by setting δ_5 and ϵ_5 as negl , and arbitrarily increasing κ , we conclude that within the UC framework, the protocol $\Pi_{\text{SSFHE}}^{\text{Bit}}$ and the ideal functionality $\mathcal{F}_{\text{CPMPC}}$ are indistinguishable.

Note on Allowing only Specified Circuits. As analyzed earlier, to improve the ϵ -correctness of Π_{SSFHE} , the parameter ϵ_0 of a given (δ_F, ϵ_F) -FHE must be sufficiently small. However, this inherently results in a slower computation speed.

If the ideal functionality $\mathcal{F}_{\text{CPMPC}}$ is modified such that the environment $\mathcal{Z}$ is restricted to executing only a predefined set of circuits $\mathcal{C}_1, \dots, \mathcal{C}_l$, then it suffices to satisfy $\epsilon_F \approx 2^{-\kappa} \max(|\mathcal{C}_1|, \dots, |\mathcal{C}_l|)^{-1}$. Thus, by reducing the number of circuits for a specific application, SSFHE can be significantly improved.

6 Conclusion

In this paper, we proposed secret-shared fully homomorphic encryption (SSFHE) that realizes circuit-private MPC. Unlike the existing LSSS-based MPC and garbled circuit-based MPC, SSFHE is the first protocol that preserves circuit privacy from third parties. Moreover, for an FHE that can perform actual gate operations, we demonstrated that SSFHE can be constructed using $O(nd^2\kappa)$ multiplications with Beaver triples over $O(\log \kappa)$ communication rounds for LWE dimension d . This protocol ensures that all n parties robustly obtain the circuit output even in the presence of an $(n - 1)$ -active static adversary.

Furthermore, the proposed approach achieves evaluation speed per gate, that is determined solely by the security parameter λ , independent of n . As a result, it offers a highly practical solution for both online communication and computation, making it a powerful technique that surpasses the limitations of the existing FHE-based MPC.

Additionally, the proposed random bit sampling protocol $\Pi_{\text{Bits}}^Q \cdot \text{BitsQ}$, which operates over composite modulus shares, and the generalized sampling protocol $\Pi_{\text{Bits}}^Q \cdot \text{SamplingQ}$ for arbitrary distributions $\mathcal{D}_{\text{poly}(\lambda)}$ are not limited to gate-operation-capable FHE. These protocols can be directly applied to other FHE schemes such as BGV/FV, and CKKS, enabling immediate construction of SSFHE. Conversely, by adopting various LSSS-based MPC schemes in constructing SSFHE, we theoretically analyzed how more efficient SSFHE schemes can be achieved in *passive* or *honest majority* settings.

Future Works. To practically implement the proposed protocol $\Pi_{\text{SSFHE}}^{\text{Bit}}$, two key aspects must be addressed:

- (i) To integrate our work with the high-speed bit-wise gate FHE method of Lee and Shin [39], it is necessary to implement a high-speed SPDZ that supports a 16-bit message space.
- (ii) It is crucial to construct various (vector) oblivious linear function evaluation (vOLE) protocols, which are essential for invoking the core ideal functionality of this paper, Dynamic SPDZ.

Acknowledgments

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. RS-2024-00409492).

References

1. Alexandru, A.: Building blocks for threshold fhe. Presentation at MPTS 2023: NIST Workshop on Multi-Party Threshold Schemes (September 2023), <https://csrc.nist.gov/csrc/media/Presentations/2023/mpts2023-day3-talk-fhe-blocks/images-media/mpts2023-3c1-slides-andreea-threshold-FHE-blocks.pdf>, accessed: 2025-01-29
2. Arratia, R., Gordon, L.: Tutorial on large deviations for the binomial distribution. *Bulletin of mathematical biology* **51**(1), 125–131 (1989)
3. Asharov, G., Jain, A., López-Alt, A., Tromer, E., Vaikuntanathan, V., Wichs, D.: Multiparty computation with low communication, computation and interaction via threshold fhe. In: *Advances in Cryptology–EUROCRYPT 2012: 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Cambridge, UK, April 15–19, 2012. *Proceedings* 31. pp. 483–501. Springer (2012)
4. Baum, C., Cozzo, D., Smart, N.P.: Using topgear in overdrive: a more efficient zkpk for spdz. In: *International Conference on Selected Areas in Cryptography*. pp. 274–302. Springer (2019)
5. Beaver, D., Micali, S., Rogaway, P.: The round complexity of secure protocols. In: *Proceedings of the twenty-second annual ACM symposium on Theory of computing*. pp. 503–513 (1990)
6. Ben-Efraim, A., Lindell, Y., Omri, E.: Efficient scalable constant-round mpc via garbled circuits. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 471–498. Springer (2017)
7. Ben-Efraim, A., Nielsen, M., Omri, E.: Turbospeedz: Double your online spdz! improving spdz using function dependent preprocessing. In: *Applied Cryptography and Network Security: 17th International Conference, ACNS 2019, Bogota, Colombia, June 5–7, 2019, Proceedings* 17. pp. 530–549. Springer (2019)
8. Bertsekas, D., Tsitsiklis, J.N.: *Introduction to probability*, vol. 1. Athena Scientific (2008)
9. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Scholl, P.: Efficient pseudorandom correlation generators: Silent ot extension and more. In: *Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III* 39. pp. 489–518. Springer (2019)
10. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Scholl, P.: Efficient pseudorandom correlation generators from ring-lpn. In: *Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part II* 40. pp. 387–416. Springer (2020)
11. Brakerski, Z., Vaikuntanathan, V.: Fully homomorphic encryption from ring-lwe and security for key dependent messages. In: *Annual cryptology conference*. pp. 505–524. Springer (2011)
12. Brenner, U., Silvanus, A., Silvanus, J.: Constructing depth-optimum circuits for adders and and-or paths. *Discrete Applied Mathematics* **310**, 10–31 (2022)

13. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*. pp. 136–145. IEEE (2001)
14. Chatel, S., Mouchet, C., Sahin, A.U., Pyrgelis, A., Troncoso, C., Hubaux, J.P.: Pelta-shielding multiparty-fhe against malicious adversaries. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. pp. 711–725 (2023)
15. Checri, M., Sirdey, R., Boudguiga, A., Bultel, J.P.: On the practical cpa d security of “exact” and threshold fhe schemes and libraries. In: *Annual International Cryptology Conference*. pp. 3–33. Springer (2024)
16. Chen, H., Chillotti, I., Song, Y.: Multi-key homomorphic encryption from tthe. In: *Advances in Cryptology–ASIACRYPT 2019: 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8–12, 2019, Proceedings, Part II 25*. pp. 446–472. Springer (2019)
17. Cheon, J.H., Choe, H., Passelègue, A., Stehlé, D., Suvanto, E.: Attacks against the ind-cpad security of exact fhe schemes. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. pp. 2505–2519 (2024)
18. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: Bootstrapping for approximate homomorphic encryption. In: *Advances in Cryptology–EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29–May 3, 2018 Proceedings, Part I 37*. pp. 360–384. Springer (2018)
19. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: *Advances in Cryptology–ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3–7, 2017, Proceedings, Part I 23*. pp. 409–437. Springer (2017)
20. Chillotti, I., Gama, N., Georgieva, M., Izabachene, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: *Advances in Cryptology–ASIACRYPT 2016: 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4–8, 2016, Proceedings, Part I 22*. pp. 3–33. Springer (2016)
21. Cohen, H.: *A course in computational algebraic number theory*, vol. 138. Springer Science & Business Media (2013)
22. Dahl, M., Demmler, D., El Kazdadi, S., Meyre, A., Orfila, J.B., Rotaru, D., Smart, N.P., Tap, S., Walter, M.: Noah’s ark: Efficient threshold-fhe using noise flooding. In: *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. pp. 35–46 (2023)
23. Damgård, I., Ishai, Y.: Scalable secure multiparty computation. In: *Annual International Cryptology Conference*. pp. 501–520. Springer (2006)
24. Damgård, I., Keller, M., Larraia, E., Pastro, V., Scholl, P., Smart, N.P.: Practical covertly secure mpc for dishonest majority–or: breaking the spdz limits. In: *Computer Security–ESORICS 2013: 18th European Symposium on Research in Computer Security, Egham, UK, September 9–13, 2013. Proceedings 18*. pp. 1–18. Springer (2013)
25. Damgård, I., Pastro, V., Smart, N., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: *Annual Cryptology Conference*. pp. 643–662. Springer (2012)

26. Dov Gordon, S., Liu, F.H., Shi, E.: Constant-round mpc with fairness and guarantee of output delivery. In: *Advances in Cryptology–CRYPTO 2015: 35th Annual Cryptology Conference*, Santa Barbara, CA, USA, August 16–20, 2015, *Proceedings, Part II* 35. pp. 63–82. Springer (2015)
27. Ducas, L., Micciancio, D.: Fhew: bootstrapping homomorphic encryption in less than a second. In: *Annual international conference on the theory and applications of cryptographic techniques*. pp. 617–640. Springer (2015)
28. Eriguchi, R., Ichikawa, A., Kunihiro, N., Nuida, K.: Efficient noise generation protocols for differentially private multiparty computation. *IEEE Transactions on Dependable and Secure Computing* **20**(6), 4486–4501 (2022)
29. Escudero, D.: An introduction to secret-sharing-based secure multiparty computation. *Cryptology ePrint Archive* (2022)
30. Escudero, D., Goyal, V., Polychroniadou, A., Song, Y., Weng, C.: Superpack: Dishonest majority mpc with constant online communication. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 220–250. Springer (2023)
31. Fouque, P.A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z., et al.: Falcon: Fast-fourier lattice-based compact signatures over ntru. *Submission to the NIST’s post-quantum cryptography standardization process* **36**(5), 1–75 (2018)
32. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: *Proceedings of the forty-first annual ACM symposium on Theory of computing*. pp. 169–178 (2009)
33. Goldwasser, S., Micali, S., Rackoff, C.: The knowledge complexity of interactive proof-systems. In: *Providing sound foundations for cryptography: On the work of shafi goldwasser and silvio micali*, pp. 203–225 (2019)
34. Goyal, V., Li, J., Misra, A.K., Ostrovsky, R., Song, Y., Weng, C.: Dishonest majority constant-round mpc with linear communication from ddh. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 167–199. Springer (2025)
35. Hazay, C., Scholl, P., Soria-Vazquez, E.: Low cost constant round mpc combining bmr and oblivious transfer. *Journal of cryptology* **33**(4), 1732–1786 (2020)
36. Keller, M., Orsini, E., Scholl, P.: Mascot: faster malicious arithmetic secure computation with oblivious transfer. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. pp. 830–842 (2016)
37. Kim, A., Lee, Y., Deryabin, M., Eom, J., Choi, R.: Lfhe: fully homomorphic encryption with bootstrapping key size less than a megabyte. *Cryptology ePrint Archive* (2023)
38. Klucznik, K.: Circuit privacy for fhew/TFHE-style fully homomorphic encryption in practice. *Cryptology ePrint Archive* (2022)
39. Lee, S., Kim, D., Shin, D.J.: Fast, compact and hardware-friendly bootstrapping in less than 3ms using multiple instruction multiple ciphertext. *Cryptology ePrint Archive* (2024)
40. Leighton, F.T.: *Introduction to parallel algorithms and architectures: Arrays· trees· hypercubes*. Elsevier (2014)
41. Li, H., Kluger, Y.: Neural inverse transform sampler. In: *International Conference on Machine Learning*. pp. 12813–12825. PMLR (2022)
42. Lindell, Y., Pinkas, B., Smart, N.P., Yanai, A.: Efficient constant-round multiparty computation combining bmr and spdz. *Journal of Cryptology* **32**, 1026–1069 (2019)

43. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: *Advances in Cryptology—EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, French Riviera, May 30–June 3, 2010. *Proceedings* 29. pp. 1–23. Springer (2010)
44. Park, J., Rovira, S.: Efficient tthe bootstrapping in the multiparty setting. *Cryptology ePrint Archive* (2023)
45. Peikert, C.: An efficient and parallel gaussian sampler for lattices. In: *Annual Cryptology Conference*. pp. 80–97. Springer (2010)
46. Rachuri, R., Scholl, P.: Le mans: Dynamic and fluid mpc for dishonest majority. In: *Annual International Cryptology Conference*. pp. 719–749. Springer (2022)
47. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)* **56**(6), 1–40 (2009)
48. Rotaru, D., Smart, N.P., Tanguy, T., Vercauteren, F., Wood, T.: Actively secure setup for spdz. *Journal of Cryptology* **35**(1), 5 (2022)
49. Turing, A.M., et al.: On computable numbers, with an application to the entscheidungsproblem. *J. of Math* **58**(345–363), 5 (1936)
50. Wang, X., Ranellucci, S., Katz, J.: Global-scale secure multiparty computation. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. pp. 39–56 (2017)
51. Wei, C., Yu, R., Fan, Y., Chen, W., Wang, T.: Securely sampling discrete gaussian noise for multi-party differential privacy. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. pp. 2262–2276 (2023)
52. Weng, C., Yang, K., Katz, J., Wang, X.: Wolverine: fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In: *2021 IEEE Symposium on Security and Privacy (SP)*. pp. 1074–1091. IEEE (2021)
53. Zhao, S., Yu, Y., Zhang, J., Liu, H.: Valiant’s universal circuits revisited: an overall improvement and a lower bound. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 401–425. Springer (2019)
54. Zhou, T., Chen, L., Che, X., Liu, W., Zhang, Z., Yang, X.: Multi-key fully homomorphic encryption scheme with compact ciphertexts. *Cryptology ePrint Archive* (2021)

Supplementary Material

In this section, we present the lemmas, proofs, and existing results that are omitted from the main body of this paper due to space constraints.

A A Brief Introduction to the Universal Composability (UC) Framework

This section provides a concise introduction to the UC framework proposed by Canetti [13]. All entities in this framework (the parties $\mathcal{P} = \{P_1, \dots, P_n\}$, the environment $\mathcal{Z}$, the adversary $\mathcal{A}$, and the trusted third party TTP) are modeled as PPT ITMs. Moreover, communication is assumed to be performed over authenticated public channels: the transmitted values (**view**) are publicly visible to everyone, but cannot be forged.

Definition 10 (Ideal Functionality $\mathcal{F}$). *An ideal functionality $\mathcal{F}$ is defined via a protocol involving n parties $P_1, \dots, P_n$, a trusted third party TTP, and an adversary $\mathcal{A}$. In this protocol, each party P_i sends input x_i to TTP. Then TTP distributes the result (possibly through $\mathcal{A}$) back to $P_1, \dots, P_n$.*

Definition 11 (UC-Realization). *The UC framework is defined by an environment $\mathcal{Z}$, an adversary $\mathcal{A}$, and parties $P_1, \dots, P_n$. The $\mathcal{Z}$ chooses one of two protocols given below with equal probability and invokes it.*

- $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\lambda, z, \text{rand})$: *Given a security parameter λ and an input z to $\mathcal{Z}$, each entity ($\mathcal{Z}$, $P_1, \dots, P_n$, and $\mathcal{A}$) receives its own randomness rand . $\mathcal{Z}$ distributes inputs to $P_1, \dots, P_n$ and $\mathcal{A}$. Then all parties and the adversary execute the protocol Π . Throughout or after execution, $\mathcal{Z}$ interacts with $\mathcal{A}$ and receives the entire **view** generated by Π via $\mathcal{A}$. Finally, $\mathcal{Z}$ outputs a single bit and terminates.*
- $\text{IDEAL}_{\mathcal{F}, \mathcal{A}, \mathcal{Z}}(\lambda, z, \text{rand})$: *Let $P_1^*, \dots, P_n^*$ be dummy parties such that each P_i^* (a deterministic ITM) simply forwards the input x_i received from $\mathcal{Z}$ to TTP. Given a security parameter λ and an input z to $\mathcal{Z}$, each entity ($\mathcal{Z}$ and $\mathcal{A}$) receives its own randomness rand . Then $\mathcal{F}$ is executed among $P_1^*, \dots, P_n^*$, TTP, and $\mathcal{A}$. Afterward, $\mathcal{A}$ interacts with $\mathcal{Z}$ to generate a (fake) **view** * , which $\mathcal{A}$ sends to $\mathcal{Z}$. Finally, $\mathcal{Z}$ outputs a single bit and terminates.*

We say that a protocol Π is a UC-realization of a ideal functionality $\mathcal{F}$ if, for every PPT environment $\mathcal{Z}$ and adversary $\mathcal{A}$, there exists an adversary $\mathcal{A}^*$ such that

$$\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}} \approx_c \text{IDEAL}_{\mathcal{F}, \mathcal{A}^*, \mathcal{Z}}.$$

If Π does not communicate with TTP, we say that Π is constructed in a plain model. Conversely, if Π internally calls an ideal functionalitis $\mathcal{F}_1, \dots, \mathcal{F}_l$ as a concurrent subroutine, then we say that Π is constructed in a $(\mathcal{F}_1, \dots, \mathcal{F}_l)$ -hybrid model. In that case, Π formally requires TTP for execution, but if $\mathcal{F}$ is already

known to be UC-realizable by some protocol Π' , then no actual TTP is needed for its real-world instantiation by replacing $\mathcal{F}$ to the Π' .

Also, we give the classification of an adversary in terms of **passive**, **active**, **static**, and **active**.

Definition 12. *An adversary $\mathcal{A}$ is called static if $\mathcal{A}$ can corrupt parties only before the protocol starts, and adaptive if $\mathcal{A}$ can corrupt parties at any arbitrary time. Moreover, $\mathcal{A}$ is called γ -passive if it can read all internal states and tapes of at most γ corrupted parties, and γ -active if it is γ -passive and can also rewrite the internal states and tapes of the corrupted γ parties.*

B Brief introduction to Dynamic SPDZ

Fig. 8 summarizes the ideal functionality $\mathcal{F}_{\text{SPDZ}}$ for Dynamic SPDZ [46].

Ideal Functionality of $\mathcal{F}_{\text{SPDZ}}$ (Dynamic SPDZ [46])	
Init:	After (Init, i, p, m) is received from all parties P_i , a key $\alpha \in \mathbb{F}_{p^m}$ is sampled and shared as $[\alpha]_i$. Then $r_1, \dots, r_m \xleftarrow{\$} \mathcal{U}_p$ are sampled, and the shares $\langle r_1 \rangle^p, \dots, \langle r_m \rangle^p$ are distributed. Initialize $\text{cnt} \leftarrow 0$ and $\text{FLAG}_{\text{abort}} \leftarrow 0$.
Rand:	Upon receiving (Rand, i) from all parties P_i , a random $\mathbf{m} \xleftarrow{\$} \mathcal{U}_p$ is stored in the cnt^{th} register. Then $\langle \mathbf{m} \rangle^p, \langle \mathbf{m}r_1 \rangle^p, \dots, \langle \mathbf{m}r_m \rangle^p$ are formed and sent, together with cnt . Finally, cnt increments.
Input:	Upon receiving (Input, $i, \mathbf{m}_i$) from all parties P_i , the value $\mathbf{m}_i$ is stored in the cnt^{th} register. Next, $\langle \mathbf{m}_i \rangle^p, \langle \mathbf{m}_i r_1 \rangle^p, \dots, \langle \mathbf{m}_i r_m \rangle^p$ are formed and sent, along with cnt . Then cnt increments.
Mul:	Upon receiving (Mul, $i, \text{cnt}_1, \text{cnt}_2$) from all parties P_i , retrieve $\mathbf{m}_1$ and $\mathbf{m}_2$ from the cnt_1^{th} and cnt_2^{th} register, respectively. Compute $\mathbf{m}_3 \leftarrow \mathbf{m}_1 \mathbf{m}_2$, store it at cnt^{th} register, and form $\langle \mathbf{m}_3 \rangle^p, \dots, \langle \mathbf{m}_3 r_m \rangle^p$. Distribute these shares and cnt , then increment cnt .
Open:	Upon receiving (Open, $i, \text{cnt}_x, \langle x^* \rangle_i^p, \dots, \langle x^* r_m^* \rangle_i^p$) from all parties P_i , retrieve $\mathbf{m}_x$ from the cnt_x^{th} register. Summations $\mathbf{m}_x^* \leftarrow \sum_{i=1}^n [\mathbf{m}_x^*]_i$, $\mathbf{m}_x^* r_j^* \leftarrow \sum_{i=1}^n [\mathbf{m}_x^* r_j^*]_i$ are computed for all j . If any mismatch with $\mathbf{m}_x$ or $\mathbf{m}_x r_j$ occurs, set $\text{FLAG}_{\text{abort}} \leftarrow 1$. Finally, send $\mathbf{m}_x^*$ to all parties.
Check:	Upon (Check, i) , if $\text{FLAG}_{\text{abort}} = 1$, then abort is output. Otherwise, new $r_1, \dots, r_m$ are sampled, the shares $\langle r_i \rangle^p$ are distributed, and reset cnt to 0.

Fig. 8: The ideal functionality for generating (Dynamic) SPDZ shares.

Details in $\mathcal{F}_{\text{SPDZ}}$. When all parties invoke $\mathcal{F}_{\text{SPDZ}}$.**Init** in Fig. 8, TTP samples a MAC key α , distributes shares $[\alpha]_i$ to the parties, and also distributes shares of uniformly random values $r_1, \dots, r_m$. Rachuri and Scholl [46] describe a scenario in which the adversary $\mathcal{A}$ may tamper with the share $\langle \mathbf{m} \rangle^p$ of a multipli-

cation result. To prevent this, they propose maintaining the additional shares $\langle \mathbf{m}r_1 \rangle^p, \dots, \langle \mathbf{m}r_m \rangle^p$.¹²

Hence, when $\mathcal{F}_{\text{SPDZ}}.\text{Input}$ is invoked for an input $\mathbf{m}$, the TTP supplies not only shares of $\mathbf{m}$ but also shares of $\mathbf{m}r_1, \dots, \mathbf{m}r_m$. The **Add** and **Mul** operations simply make TTP add or multiply the register values, then distribute updated shares. During **Open**, if any party reports shares inconsistent with TTP’s computation, $\text{FLAG}_{\text{abort}}$ is set to 1. Finally, $\mathcal{F}_{\text{SPDZ}}.\text{Check}$ aborts the protocol if $\text{FLAG}_{\text{abort}} = 1$ and otherwise, it resamples $r_1, \dots, r_m$.

C Rationale for Choosing the Moduli p_1, q_2, q_3 , and p_{res} .

Note that in the proposed protocol $\Pi_{\text{SSFHE}}^{\text{Bits}}$, the integer moduli p_1, q_2, q_3 , and p_{res} are chosen differently. The modulus p_1 is used as a parameter to minimize the modulus of ciphertexts before performing homomorphic operation in GINX FHE. Thus, in this paper, we select p_1 as a prime number to efficiently generate the random samples required for encryption.

The modulus p_{res} is used in the proposed protocol Dec^{new} . It is significantly smaller than p_1 and is also chosen as a prime number to ensure the efficient execution of Dec^{new} .

The modulus q_2 is used in the computation process of GINX FHE. To achieve fast operations, q_2 is typically chosen as a prime number with $q_2 \approx 2^{32}$ or $q_2 \approx 2^{64}$, or as a composite number. To accommodate the use of composite numbers, we usually design q_2 as a composite number consisting of the product of NTT-friendly prime numbers.

Finally, the modulus q_3 is used in the existing smudging noise-based protocol Dec^{prev} . Since q_3 must be large enough to maintain security, it is chosen such that $q_3 > 2^{80}$. Thus, q_3 is also designed as a composite number consisting of the product of NTT-friendly prime numbers.

D Concrete Analysis of the Number of Calls to $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ while Running $\Pi_{\text{SSFHE}}^{\text{Bit}}.\text{KeyGen}$.

Given a security parameter $\lambda \approx 128$, the parameters l, l_1, l_2, l_3 , and σ are chosen to be small in order to minimize the error probability in the homomorphic operations of the given FHE scheme [20, 39]. For simplicity, we consider these parameters as constants relative to λ .

In Theorem 2, $f_{\text{com}}(\lambda)$ corresponds to the key size of the homomorphic encryption scheme, which is practically large, satisfying $f_{\text{com}}(128) \geq 2^{20}$ for $\lambda = 128$ [20, 39]. Additionally, for an asymptotic analysis in practical settings, let d represent the dimension of the ciphertext, assuming $N_1 \approx N_2 \approx N_3 = O(d)$.

¹²Rachuri and Scholl [46] originally used a single random r ; however, Lemma 1 of [46] shows that this approach only guarantees a tampering probability of $O(p^{-1})$. To achieve $O(p^{-m})$, one can “boost” this soundness using multiple random values $r_1, \dots, r_m$.

For $\kappa \approx 40$, we have $f_{\text{com}}(\lambda) \gg \kappa^2$, and since $f_{\text{com}} = O(d^2)$, it follows that $f_{\text{com}} \gg ld_1\sigma^2$. Thus, the Theorem 2 imply that when using a discrete Gaussian as the error distribution, $\mathcal{F}_{\text{SPDZ}}.\mathbf{Mul}$ is called $f_{\text{com}}(\lambda)\kappa = O(d^2\kappa)$ times. If a binomial distribution is used instead, $\mathcal{F}_{\text{SPDZ}}.\mathbf{Mul}$ is called $f_{\text{com}}(\lambda) = O(d^2)$ times¹³. Moreover, the number of communication rounds required is $O(\log \kappa)$ when using a discrete Gaussian and $O(1)$ when using a binomial distribution.

Additionally, according to Proposition 1-(ii) in existing Dynamic SPDZ results, the seed communication required for generating Beaver triples for multiplication can be achieved with only $O(n \log d + n \log \kappa)$ and $O(n \log d)$, repectively.

E Useful lemmas and proofs

Lemma 6 (Blelloch’s Prefix Sum Algorithm [40]). *For a binary operation $\cdot$ on l elements $a_1, \dots, a_l$, there is an arithmetic circuit $\mathcal{C}$ that outputs $a_1, a_1 \cdot a_2, \dots, a_1 \cdot \dots \cdot a_l$ using at most $2l - 2$ operations and with depth at most $2\lceil \log l \rceil$. Moreover, there is a circuit $\mathcal{C}$ that outputs only $a_1 \cdots a_l$ using $l - 1$ operations with depth at most $\lceil \log l \rceil$.*

The proof of Lemma 6 is constructed by presenting a concrete circuit and analyzing its depth and the number of actual operations performed. For the detailed proof, refer to [40].

Lemma 7 (Random Smudging Lemma). *Let $X \xleftarrow{\$} \mathcal{U}_{2^\kappa}$ and $Y \xleftarrow{\$} \mathcal{D}_B$ such that X and Y are independent, and let $\mathcal{E}$ be the event such that $B - 1 \leq X + Y < 2^\kappa$, with $\mathcal{E}^c$ being its complement. Then the following properties hold:*

- (i) $X + Y$ and Y are independent conditioned on $\mathcal{E}$.
- (ii) $\Pr[\mathcal{E}^c] \leq (B - 1)2^{-\kappa}$.
- (iii) $\Delta(X + Y, Y) \leq (B - 1)2^{-\kappa}$.

Lemma 7 extends a well-known result used in FHE-based MPC to demonstrate the stability of the smudging technique (see Lemma 2.1 in [3]). While previous works considered the case where the concealed value Y was deterministic, we generalize such analysis to the randomized values of Y and provide various properties and proofs.

proof of Lemma 7.

- (i). We prove that for any $B - 1 \leq i < 2^\kappa$,

$$\Pr[X + Y = i \mid \mathcal{E}] = \Pr[X + Y = i \mid \mathcal{E}, Y = j].$$

¹³If the recent technique with lightweight ev keys [37] is used to construct $\mathbf{SSFHE}$ while the evaluation speed of $\mathbf{Eval}^F$ for each party may slow down, the number of calls to $\mathcal{F}_{\text{SPDZ}}.\mathbf{Mul}$ within $\Pi_{\text{SSFHE}}^{\text{Bit}}.\mathbf{KeyGen}$ for $\mathcal{DG}_\sigma$ and $\mathcal{B}_\sigma$ are expected to decrease from $O(d^2\kappa)$ and $O(d^2)$ to $O(d^{2-c}\kappa)$ and $O(d^{2-c})$, respectively, for some constant $c > 0$.

The pairs (X, Y) that satisfy $X + Y = i$ are given by $(i, 0), (i - 1, 1), \dots, (i - B + 1, B - 1)$. Hence, we have

$$\Pr[X + Y = i \mid \mathcal{E}] = 2^{-\kappa} \sum_{j=0}^{B-1} \Pr[Y = j] = 2^{-\kappa}.$$

If the value of $Y = j$ is known, then

$$\Pr[X + Y = i \mid \mathcal{E}, Y = j] = \Pr[X = i - j] = 2^{-\kappa}.$$

Thus, $X + Y$ and Y are independent given $\mathcal{E}$.

(ii). Define $\mathcal{G}_i \triangleq \{(X, Y) \in \mathcal{E}^c \mid X = i\}$. For any $(i, y) \in \mathcal{G}_i$, the possible values of Y are:

$$Y = \underbrace{0, 1, \dots, B - 2 - i}_{B-1-i}, \underbrace{2^\kappa - i, \dots, 2^\kappa - 1}_i.$$

Since there are exactly $B - 1$ such values, we obtain:

$$\begin{aligned} \Pr[\mathcal{E}^c] &= \sum_{i=0}^{B-1} \Pr[\mathcal{G}_i] \leq \sum_{i=0}^{B-1} \sum_{(i, Y_j) \in \mathcal{G}_i} \Pr[X = i] \Pr[Y = Y_j] \\ &= \sum_{i=0}^{B-1} \sum_{(i, Y_j) \in \mathcal{G}_i} \Pr[X = i] 2^{-\kappa} \\ &= 2^{-\kappa} \sum_{i=0}^{B-1} \Pr[X = i] (B - 1) = (B - 1) 2^{-\kappa}. \end{aligned}$$

(iii). From (i), we obtain $\Pr[X + Y = i] - \Pr[Y = i] = 0$ for all $B - 1 \leq i < 2^\kappa$. hence, we derive:

$$\begin{aligned} \Delta(X + Y, Y) &= 2^{-1} \left[\sum_{i=0}^{B-2} (\Pr[Y = i] - \Pr[X + Y = i]) \right. \\ &\quad \left. + \sum_{i=2^\kappa}^{2^\kappa + B - 2} \Pr[X + Y = i] \right] \\ &\leq 2^{-1} \sum_{i=0}^{B-2} \Pr[Y = i] + 2^{-1} \sum_{i=0}^{B-2} (\Pr[X + Y = i] + \Pr[X + Y = i + 2^\kappa]) \\ &= (B - 1) 2^{-\kappa-1} + 2^{-1} \Pr[\mathcal{E}^c] = (B - 1) 2^{-\kappa}. \end{aligned}$$

Thus, we conclude that $\Delta(X + Y, Y) \leq (B - 1) 2^{-\kappa}$. $\square$

Lemma 8. For a given prime field $\mathbb{Z}_p$, an integer $n > 0$, and random bits $a_1, \dots, a_n$ and $b_1, \dots, b_n, c$, where $a_1, \dots, a_n$ are mutually independent and also independent of $b_1, \dots, b_n, c$, while $b_1, \dots, b_n$ and c may be dependent. Moreover, assume that there exists at least one nonzero element $b_i \in \mathbb{Z}_p$. Then, the following inequality holds:

$$\Pr \left[\sum_{i=1}^n a_i b_i = c \right] \leq 2^{-1}.$$

proof of Lemma 8.

We first consider the case when $p = 2$. Define the function $L : \mathbb{Z}_2^n \rightarrow \mathbb{Z}_2$ by

$$L(a_1, \dots, a_n) = \sum_{i=1}^n a_i b_i.$$

Since L is a linear map, the rank-nullity theorem from linear algebra implies that $\dim(\text{Ker}(L)) = n - 1$. Therefore, we have $|L^{-1}(0)| = |L^{-1}(1)| = 2^{n-1}$, meaning that

$$\Pr \left[\sum_{i=1}^n a_i b_i = c \right] = 2^{-1}.$$

Thus, we now consider the case when $p \neq 2$.

First, we verify the case when $n = 1$. If $c = 0$, then:

$$\Pr[a_1 b_1 = 0] = \Pr[a_1 = 0] = 2^{-1}.$$

If $c \neq 0$, then:

$$\Pr[a_1 b_1 = c] = \Pr[a_1 = 1 \wedge b_1 = c] \leq \Pr[a_1 = 1] = 2^{-1}.$$

Thus, the statement holds for $n = 1$.

Now, assuming that the statement holds for n , we prove the case for $n + 1$.

If $c = 0$, we have:

$$\begin{aligned} & \Pr[a_{n+1} b_{n+1} + \sum_{i=1}^n a_i b_i = 0] \\ &= \Pr[a_{n+1} = 0] \cdot \Pr \left[\sum_{i=1}^n a_i b_i = 0 \right] + \Pr[a_{n+1} = 1] \cdot \Pr \left[\sum_{i=1}^n a_i b_i = -b_{n+1} \right] \\ &\leq 2^{-1} \cdot 2^{-1} + 2^{-1} \cdot 2^{-1} = 2^{-1}. \end{aligned}$$

If $c \neq 0$, we have:

$$\Pr[a_{n+1} b_{n+1} + \sum_{i=1}^n a_i b_i = c]$$

$$\begin{aligned}
&= \Pr[a_{n+1} = 0] \cdot \Pr\left[\sum_{i=1}^n a_i b_i = c\right] + \Pr[a_{n+1} = 1] \cdot \Pr\left[\sum_{i=1}^n a_i b_i = c - b_{n+1}\right] \\
&\leq 2^{-1} \cdot 2^{-1} + 2^{-1} \cdot 2^{-1} = 2^{-1}.
\end{aligned}$$

Thus, the statement holds for all n by induction. $\square$

proof of Lemma 3.

Since $\Psi^p(X)$ is a polynomial of degree at most $p - 1$, given $\mathbf{m} \in \mathbb{Z}_p$, we can compute $\mathbf{m}^1, \mathbf{m}^2, \dots, \mathbf{m}^{p-1}$ and obtain $\Psi(\mathbf{m})^p$ by taking their linear combination. From Lemma 6, we conclude that there exists a circuit $\mathcal{C}_{\Psi}^p$ with the multiplication count of at most $p - 2 \leq p$ and a multiplication depth of at most $\lceil \log(p - 1) \rceil \leq \lceil \log p \rceil$.

Similarly, since $\Psi_{B,j}^p(X)$ is a polynomial of degree at most $B - 1$, Lemma 6 ensures the existence of a circuit $\mathcal{C}_{\Psi_{B,j}}^p$ with the multiplication count of at most $B - 2 \leq B$ and the multiplication depth of at most $\lceil \log(B - 1) \rceil \leq \lceil \log B \rceil$. $\square$

Proof of Proposition 2.

First, define the approximated distribution $\mathcal{D}'$ as follows:

$$\mathcal{D}'(i) \triangleq \begin{cases} \frac{\mathcal{D}(i)}{C}, & \text{if } -B/2 < i < B/2, \\ 0, & \text{otherwise.} \end{cases}$$

where C is the normalization constant given by

$$C \triangleq \sum_{-B/2 < j < B/2} \mathcal{D}(j).$$

Additionally, $C = \Pr[|x| < B/2]$ holds. Thus, we obtain:

$$\begin{aligned}
\Delta(\mathcal{D}, \mathcal{D}') &= 0.5 \sum_{i \in \mathbb{Z}} |\mathcal{D}(i) - \mathcal{D}'(i)| \\
&= 0.5 \left(\sum_{|i| \geq B/2} \mathcal{D}(i) + \sum_{-B/2 < i < B/2} (\mathcal{D}'(i) - \mathcal{D}(i)) \right) \\
&= 0.5(1 - C + 1 - C) = 1 - C = 1 - \Pr_{x \leftarrow \mathcal{D}}[|x| < B/2].
\end{aligned}$$

Next, define another approximated distribution $\mathcal{D}''(i)$ as follows:

$$\mathcal{D}''(i) \triangleq \begin{cases} \lfloor \mathcal{D}'(i) 2^\kappa \rfloor 2^{-\kappa}, & \text{if } -B/2 < i < B/2, \\ \nu \triangleq \sum_{-B/2 < i < B/2} (\mathcal{D}'(i) - \lfloor \mathcal{D}'(i) 2^\kappa \rfloor 2^{-\kappa}), & \text{if } i = -B/2, \\ 0, & \text{otherwise.} \end{cases}$$

Here, $\nu = 1 - \sum_{-B/2 < i < B/2} \mathcal{D}''(i)$, and since the right-hand side can be computed exactly using κ bits for the case of $-B/2 < j < B/2$, ν can also be represented exactly using κ bits, satisfying $\nu \leq 2(B/2 + 1)2^{-\kappa}$. Moreover, we obtain:

$$\Delta(\mathcal{D}', \mathcal{D}'') = 0.5 \sum_{i \in \mathbb{Z}} |\mathcal{D}'(i) - \mathcal{D}''(i)|$$

$$\begin{aligned}
&= 0.5 \sum_{-B/2 < i < B/2} (\mathcal{D}''(i) - \mathcal{D}'(i)) + 0.5\nu \\
&\leq \nu \leq 2(B/2 + 1)2^{-\kappa} = (B + 2)2^{-\kappa}.
\end{aligned}$$

Finally, defining $\mathcal{D}^*(i) \triangleq \mathcal{D}''(i - B/2)$, we obtain:

$$\Delta(\mathcal{D}, \mathcal{D}^* - C) \leq 1 - \Pr_{x \leftarrow \mathcal{D}}[|x| < B/2] + (B + 2)2^{-\kappa}.$$

Since $\text{supp}(\mathcal{D}^*) \subset \{0, \dots, B - 1\}$, the proof is complete. $\square$

proof of Lemma 4.

We construct the circuit $\mathcal{C}_{\text{comp}}^p$ as follows:

(Depth 1): For all $i = 1, \dots, \kappa$, compute $a_i \leftarrow \neg a_i$ without multiplication. Then, compute $Y_i \leftarrow a_i \oplus b_i$ and $X_i \leftarrow a_i \wedge b_i$. From Eq. (10), a total of $\kappa + \kappa = 2\kappa$ multiplications over $\mathbb{Z}_p$ are performed.

(Depth 2 - Depth $2\lceil \log \kappa \rceil + 1$): For all $j = 1, \dots, \kappa$, compute $Z_j \leftarrow Y_j \wedge Y_{j+1} \wedge \dots \wedge Y_\kappa$. By Lemma 6, this requires a depth of at most $2\lceil \log \kappa \rceil$ and at most $2\kappa - 2$ multiplications over $\mathbb{Z}_p$.

(Depth $2\lceil \log \kappa \rceil + 2$): For all $j = 1, \dots, \kappa$, compute $Z_j \leftarrow X_j \wedge Z_j$. From Eq. (10), at most κ multiplications over $\mathbb{Z}_p$ are performed.

(Depth $2\lceil \log \kappa \rceil + 3$ - Depth $3\lceil \log \kappa \rceil + 2$): Compute $Z_1 \vee \dots \vee Z_\kappa$. By Lemma 6, this requires a depth of at most $\lceil \log \kappa \rceil$ and at most $\kappa - 1$ multiplications over $\mathbb{Z}_p$.

Thus, the circuit $\mathcal{C}_{\text{comp}}^p$ executes at most $3\lceil \log \kappa \rceil + 2$ depth with at most $2\kappa + 2\kappa - 2 + \kappa + \kappa - 1 \leq 6\kappa$ multiplications over $\mathbb{Z}_p$. $\square$

proof of Lemma 5.

When executing $\Pi_{\mathcal{D}.\text{Bits}}$ in parallel $B\kappa$ times, $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ is invoked κ times within a single communication round. Subsequently, executing $\mathcal{C}_{\text{comp}}^p$ for B instances in parallel requires at most $\text{depth}(\mathcal{C}_{\text{comp}}^p) \leq 3\lceil \log \kappa \rceil + 2$ communication rounds and at most $B|\mathcal{C}_{\text{comp}}^p| \leq 6B\kappa$ calls to $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$, according to Lemma 4.

Thus, $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ is invoked at most $(6B + 1)\kappa \leq 7B\kappa$ times over a maximum of $3\lceil \log \kappa \rceil + 3$ communication rounds. $\square$

proof of Proposition 3.

Before proceeding with the proof, let us assume that the secret shares $[m]_1, \dots, [m]_n$ of a random bit m are given. Define $\Delta = \lceil p/n \rceil$, and select values l_i and h_i such that

$$[m]_i = h_i \Delta + l_i, \quad 0 \leq l_i < \Delta.$$

Note that given $[m]_1, \dots, [m]_{n-1}$ and m , the value of $[m]_n$ is uniquely determined.

We analyze the following two cases:

- (a) The number of cases that $\sum [m]_i - \eta p_0 \in \mathbb{Z}$ does not equal to $m \in \mathbb{Z}$, where $\eta \leftarrow \lceil (\Delta \sum_{i=1}^n h_i) p_0^{-1} \rceil$.

- (b) The necessary condition for the $\mathbf{m}$'s distribution to be non-uniform given a fixed $(h_1, \dots, h_n)$.

For the case (a), the analysis follows the approach of [48]. Given the condition $0 \leq \sum_{i=1}^n l_i < p_0$, we obtain the following relation:

$$\eta = \Delta \left(\sum_{i=1}^n h_i \right) p_0^{-1} + \left(\sum_{i=1}^n l_i \right) p_0^{-1} - \mathbf{m} p_0^{-1}. \quad (14)$$

Thus, the only case when $\eta \neq \lceil \Delta \left(\sum_{i=1}^n h_i \right) p_0^{-1} \rceil$ is when $\sum_{i=1}^n l_i < \mathbf{m}$, which occurs only if $\mathbf{m} = 1$ and $l_1 = \dots = l_n = 0$.

Next, we analyze the case (b). Assuming the honest party has index 1, we determine the necessary condition under which the adversary gains information about $\mathbf{m}$ from the publicly available $h_1, \dots, h_n$. The adversary knows $\vec{h} = (h_1, \dots, h_n)$, $\vec{l}^* = (l_2, \dots, l_n)$, and the public parameters Δ, p_0 . For a conservative analysis, assume that the adversary also knows the exact value of η . Define the function:

$$\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}(l_1) = \Delta \left(\sum_{i=1}^n h_i \right) + \left(\sum_{i=1}^n l_i \right) - \eta p_0 \in \mathbb{Z}.$$

If the preimage of 0 $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(0) \neq \emptyset$ and the preimage of 1 $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(1) \neq \emptyset$ both hold, then although the adversary knows that l_1 belongs to one of these two nonempty preimages $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(0)$ and $\in \pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(1)$, it does not gain any information about $\mathbf{m}$.

However, if either $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(0) = \emptyset$ or $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(1) = \emptyset$, the adversary immediately learns the exact value of $\mathbf{m}$. Thus, analyzing the conditions under which such cases occur is essential.

Case 1. Suppose that $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(0) = \emptyset$. Then, we must always select $l_1 \in \pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(1)$. If $l_1 \neq 0$, then $l_1 - 1 \in \pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(0)$, which leads to a contradiction. Thus, such an l_1 cannot be an element of $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(1)$. Now, consider the case when $l_1 = 0$. For there to exist some $l'_1 \in \pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(0)$, there must exist some integer k such that $l'_1 = l_1 - 1 + p_0 k$. However, since no such value satisfies $0 \leq l'_1 < \Delta$, we conclude that $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(0) = \emptyset$.

Case 2. Conversely, suppose that $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(1) = \emptyset$. Then, we must always select $l_1 \in \pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(0)$. If either $l_1 \neq \Delta - 1$ or $h_1 \Delta + l_1 \neq p_0 - 1$, then $l_1 + 1 \in \pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(1)$, which leads to a contradiction. Thus, such an l_1 cannot be an element of $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(0)$. Now, consider the case when either $l_1 \neq \Delta - 1$ or $h_1 \Delta + l_1 \neq p_0 - 1$. If there exists some $l'_1 \in \pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(1)$, then there must exist some integer k such that $l'_1 = l_1 + 1 + p_0 k$. However, since no such value satisfies $0 \leq l'_1 < \Delta$, we conclude that $\pi_{\vec{h}, \vec{l}^*, p_0, \eta, \Delta}^{-1}(1) = \emptyset$.

From **Case 1** and **Case 2**, we define the set of messages that result in leakage as follows:

$$\text{SetB}_{p_0, \Delta} \triangleq \left\{ [m]_i \in \mathbb{Z}_{p_0} \mid \{ \lfloor [m]_i / \Delta \rfloor \in \{0, \Delta - 1\} \} \vee \{ [m]_i = p_0 - 1 \} \right\}. \quad (15)$$

We observe that $|\text{SetB}_{p_0, \Delta}| \leq 2(n+1)$. Now, we proceed to prove the following statements (i), (ii), and (iii).

(i). If for any party P_i , the share $[m]_i$ satisfies $[m]_i \in \text{SetB}_{p_0, \Delta}$, then the share is broadcasted and discarded. Thus, the event in which a share is discarded corresponds to $\bigcup_{i=1}^n \{ [m]_i \in \text{SetB}_{p_0, \Delta} \}$. The probability of each such event is given by:

$$\Pr[[m]_i \in \text{SetB}_{p_0, \Delta}] \leq \frac{2(n+1)p_0^{n-2}}{p_0^{n-1}} = 2(n+1)p_0^{-1}.$$

By applying the union bound, we obtain:

$$\Pr\left[\bigcup_{i=1}^n \{ [m]_i \in \text{SetB}_{p_0, \Delta} \}\right] \leq \sum_{i=1}^n \Pr[[m]_i \in \text{SetB}_{p_0, \Delta}] \leq 2n(n+1)p_0^{-1} < 1.$$

At this point, in case (a) of Π_{Bits}^Q , the event in which **abort** occurs is when at least α samples out of $t + \kappa\kappa' + \alpha$ are discarded. Now, consider the case where $\alpha = 0$. In this case, the number of generated samples is $t + \kappa'\kappa$, so if even a single share among them belongs to $\text{SetB}_{p_0, \Delta}$, the protocol immediately executes **abort**. Thus, applying the union bound again, $\text{prob}_1 \leq 2(t + \kappa'\kappa)n(n+1)p_0^{-1}$.

Now, consider the case where $\alpha > 2n(n+1)(t + \kappa\kappa')(p_0 - 2n(n+1))^{-1}$. This is equivalent to $\alpha(t + \kappa\kappa' + \alpha)^{-1} > 2n(n-1)p_0^{-1}$. then we obtain:

$$\text{prob}_1 \leq \sum_{i=\alpha}^{t+\kappa\kappa'+\alpha} \binom{t+\kappa\kappa'+\alpha}{i} [2(n+1)p_0^{-1}]^i [1 - 2(n+1)p_0^{-1}]^{t+\kappa\kappa'+\alpha-i}.$$

By applying Chernoff bound [2], if $\alpha(t + \kappa\kappa' + \alpha)^{-1} > 2n(n-1)p_0^{-1}$, the probability of this happening is:

$$\log \text{prob}_1 \leq -(t + \kappa\kappa' + \alpha) \text{KL}\left(\alpha(t + \kappa\kappa' + \alpha)^{-1} \parallel 2n(n+1)p_0^{-1}\right).$$

(ii). Now, we compute an upper bound for probability prob_2 . Since at least $\langle \mathbf{u}_{ji} \rangle^{p_0}$ in protocol RepQ are guaranteed to be random bits, we utilize the additional κ' bits $\langle \mathbf{u}_1 \rangle^{p_0}, \dots, \langle \mathbf{u}_{\kappa'} \rangle^{p_0}$ to compute the sum:

$$S^{p_0} \leftarrow \left[\sum_{i=1}^t r_i \mathbf{m}_i \right] + \left[\sum_{i=1}^{\kappa'} \mathbf{u}_i 2^{i-1} \right] \in \mathbb{Z}.$$

Since $0 \leq S^{p_0} < t + 2^{\kappa'}$, opening $\langle S^{p_0} \rangle^{p_0}$ accurately yields the correct S^{p_0} when $p_0 > t + 2^{\kappa'}$. From Lemma 7-(i), when $t \leq S^{p_0} < 2^{\kappa'}$, there is no information leakage about $\sum_{i=1}^t \mathbf{m}_i$. By Lemma 7-(ii) in Supplementary Material, the probability that $S^{p_0} < t$ or $S^{p_0} \geq 2^{\kappa'}$ is $t2^{-\kappa'}$, which corresponds to the event in which

the Protocol RepQ invokes the Restart command. Since this process repeats κ times, by union bound, the probability that Restart is invoked at least once is $\text{prob}_2 \leq \kappa t 2^{-\kappa'}$.

(iii). Now, when the protocol operates correctly, there are two cases in which the adversary's attack may go undetected: (a) According to the definition of LSSS-based MPC, there is a probability of $2^{-\Theta(\kappa)}$ that at least one of the opened values contains an incorrect value; (b) Assuming that all opened values are correct, the adversary may manipulate the corrupted party to generate a share $\langle \mathbf{m}' \rangle^{p_i}$ for some $\mathbf{m} \neq \mathbf{m}'$. Suppose the adversary selects $b_1, \dots, b_t$ and translates their share to construct $\langle \mathbf{m}_i^* \leftarrow \mathbf{m}_i + b_i \rangle^{p_j}$.

A critical observation here is that the bit shares used for smudging noise can also be altered, such that

$$\langle \mathbf{u}_i^* \leftarrow \mathbf{u}_i^* + c_i \rangle^{p_j}.$$

Moreover, it is possible to manipulate them as follows:

$$\langle \mathbf{u}_1^* \leftarrow \mathbf{u}_1^* + 2 \rangle^{p_j}, \quad \langle \mathbf{u}_1^* \leftarrow \mathbf{u}_1^* - 1 \rangle^{p_j}.$$

This manipulation prevents $\mathbf{u}_i^*$ from being random bits while ensuring that the sum $\langle E \rangle^{p_j}$ remains unchanged. However, since the protocol only uses the randomly generated samples $\langle \mathbf{m}_1^* \rangle^{p_j}, \dots, \langle \mathbf{m}_t^* \rangle^{p_j}$, any undetected modifications to $\langle \mathbf{u}_i^* \leftarrow \mathbf{u}_i^* + c_i \rangle^{p_j}$ do not impact the protocol's correctness.

Now, if the smudging noise is modified as $\langle E^* \leftarrow E - c \rangle^{p_j}$, the condition for the protocol to avoid triggering abort is given by:

$$S^{p_0} = S^{p_0} + \sum_{i=1}^t r_i b_i - c \bmod p_j \Leftrightarrow \sum_{i=1}^t r_i b_i = c \bmod p_j.$$

Since at least one of $b_1, \dots, b_t$ is a nonzero unit, by Lemma 8, the probability of this condition holding is at most 2^{-1} . Since this is repeated κ times, the probability of successfully bypassing detection is at most $2^{-\kappa}$.

Thus, apart from a total probability of $2^{-\Theta(\kappa)}$, all adversary attacks are detected, and all resulting distributed bits are perfectly uniformly random. $\square$

proof of Lemma 4.

(i). The only instance that $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ is invoked within $\Pi_{\text{Bits}}^Q.\text{BitsQ}$ is during the parallel execution of $\Pi_{\text{Bits}}^p.\text{Bits}$ for a prime p_0 , which occurs $2t + \kappa^2$ times.

Finally, we verify that the probability of detecting an adversary's attack, given the provided parameters, is $2^{-\Theta(\kappa)}$. If $t = 2^{o(\kappa)}$ samples are generated, from Proposition 3-(i), we obtain $\text{prob}_1 \leq (t + \kappa^2)n(n+1)q_0^{-1} = 2^{-\Theta(\kappa)}$. Then from Proposition 3-(ii),

$$\text{prob}_2 \leq 2^{-\kappa + \log \kappa + O(\kappa)} = 2^{-\Theta(\kappa)}.$$

Thus, by combining this result with Proposition 3-(iii), we conclude that, except for a probability of $2^{-\Theta(\kappa)}$, the output consists of uniformly random bits.

(ii). Within $\Pi_D^Q.\text{SamplingQ}$, $\Pi_{\text{Bits}}^Q.\text{BitsQ}$ is first invoked in parallel $t\kappa = 2^{o(\kappa)}$ times to generate shares for random bits. According to Lemma 4, this requires at most $t\kappa + \kappa^2$ calls to $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ in a single round.

Next, for each prime of $p_1, \dots, p_l$, the **compare** function is invoked in parallel tB times over κ -bit shares. By Lemma 4, this process is executed over a maximum of $\max_{s=1, \dots, l} \text{depth}(\mathcal{C}_{\text{comp}}^{p_s}) = 3\lceil \log \kappa \rceil + 2$ rounds and requires at most $\sum_{s=1}^l |\mathcal{C}_{\text{comp}}^{p_s}| \leq 6tB\kappa$ multiplications.

Thus, $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ is invoked at most $3\lceil \log \kappa \rceil + 3$ rounds, with at most $(6lB + 1)t\kappa + \kappa^2 \leq 7lBt\kappa + \kappa^2$ calls in total. $\square$

Proof of Theorem 1.

We first analyze the correctness of SSFHE.

Analysis of ϵ_S -correctness of SSFHE. Given that SSFHE executes without an **abort**, the output being incorrect can only occur in one of the following two cases:

- (i) According to the definition of LSSS-based MPC, there is a probability of $2^{-\Theta(\kappa)}$ that at least one of the opened values is incorrect.
- (ii) The ciphertext resulting from an **Eval** operation in FHE contains an incorrect message value.

Given an ϵ_F -correct FHE and a circuit $\mathcal{C}$ with a polynomial number of gates $\text{gate}(\mathcal{C}) = \text{poly}(\lambda)$, the upper bound on errors occurring during computation is $|\mathcal{C}|\epsilon_F$.

By applying the union bound to these two cases, we obtain $\epsilon_S \leq O(2^{-\kappa}) + \epsilon_F \text{poly}(\lambda)$ ^{14 15}.

Analysis of δ_S -IND-CPA security of SSFHE.

Next, we analyze the IND-CPA security of the proposed SSFHE. That is, we examine the information leakage occurring during the opening of $\langle \text{pk} \rangle$, $\langle \text{ev} \rangle$, and the ciphertext $\langle \text{ct} \rangle$ encrypting an arbitrary message.

Since, by definition, κ -soundness LSSS-based MPC ensures that **view** is independent of the message, no additional leakage occurs beyond the information explicitly revealed by the message itself. Thus, all instances of **view** consist of pk , ev , ct , and an independent random source. Therefore, given a δ_F -IND-CPA secure FHE, the constructed SSFHE inherits δ_F -IND-CPA security. $\square$

¹⁴Note that since circuit $\mathcal{C}$ consists of $\text{poly}(\lambda)$ gates, for $\epsilon_F = 2^{-\Theta(\kappa)}$, we have asymptotically $\text{gate}(\mathcal{C})\epsilon_F = 2^{\Theta(-\kappa)}$. However, in practical parameter selection, if we use the heuristic assumption $\text{poly}(\lambda) \leq 2^{80}$, then we require an FHE satisfying $\epsilon_F \approx 2^{-\kappa-80}$ -correctness.

¹⁵Recently, Chen et al. [17] proposed a method to recover the secret key of gate-based FHE by exploiting trivial operations such as $\text{AND}(\text{ct}, \text{ct})$, $\text{XOR}(\text{ct}, \text{ct})$, and $\text{OR}(\text{ct}, \text{ct})$ on a ciphertext ct encrypting a bit message m . From the perspective of this paper, this phenomenon arises because trivial circuit operations are permitted, leading to an increase in the effective ϵ_F value of the given FHE. Thus, when evaluating a given circuit, such trivial operations must be removed to minimize ϵ_F .

proof of Theorem 2.

The shares generated in **KeyGen** are as follows:

$$[[\vec{s}_1]]^Q, [[\vec{s}_2]]^Q, [[\vec{s}_3]]^Q, [[\vec{e}]]^Q, [[f_{i_1 j_2}]]^Q, [[\vec{g}_{i_2 j_2}]]^Q, [[\vec{h}_{i_3 j_3}]]^Q.$$

First, assume that $[[\vec{s}_1]]^Q$, $[[\vec{s}_2]]^Q$, and $[[\vec{s}_3]]^Q$ have already been obtained. Subsequently, functions **OneHot** and **IndVec**, which take these shares as inputs, must be retrieved via the protocol. Since **OneHot** is a projection map, it does not require communication. However, **IndVec** _{i,z} must evaluate the function **Equal**, which is computed using the circuit $\mathcal{C}_{\psi_{B,j}}^p$ guaranteed by Lemma 3. Thus, for all $i = 1, \dots, N_1$ and $z = O(\sigma)$, the protocol $\Pi_\psi.\text{Ind}$ in Fig. 3 is invoked in parallel a total of $O(N_1 \sigma)$ times. From Lemma 3, this results in $O(\log \sigma)$ rounds and at most $O(lN_1 \sigma^2)$ calls to $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$.

(i). Consider the case where $\mathcal{D}_B \leftarrow \mathcal{DG}_{O(\sigma)}$. In this case, $\Pi_{\text{Bits}}^Q.\text{SamplingQ}$ is invoked

$$N_1 + N_2 + N_3 + d_1 \mathbf{v} + l_1 d_1 N_2 + (l_2 d_2 + l_3 d_3) N_1$$

times. By combining Proposition 4-(ii) with the round and invocation analysis for **IndVec**, we conclude that $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ is called at most $O((f_{\text{com}}(\lambda)\kappa + \kappa^2 + lN_1 \sigma^2))$ times while at most $O(\log \kappa + \log \sigma)$ communication rounds.

(ii). Now, consider the case where $\mathcal{D}_B \leftarrow \mathcal{DG}_{O(\sigma)}$. Here, $\Pi_{\text{Bits}}^Q.\text{BitsQ}$ is invoked

$$\sigma(N_1 + N_2 + N_3 + d_1 \mathbf{v} + l_1 d_1 N_2 + (l_2 d_2 + l_3 d_3) N_1)$$

times. Using Proposition 4-(i) and the round and invocation analysis for **IndVec**, we conclude that $\mathcal{F}_{\text{SPDZ}}.\text{Mul}$ is called at most $O((f_{\text{com}}(\lambda) + \kappa^2 + lN_1 \sigma^2))$ times while at most $O(\log \sigma)$ communication rounds. $\square$

F Proof of Theorem 3

Before proving Theorem 3, we introduce the projection distinguisher and necessary lemmas. In particular, the projection distinguisher provides a sufficient condition for when $(A, X) \cong_c (B, Y)$ holds given that $X \cong_c Y$.

Definition 13 (Projection distinguisher $\mathfrak{D}_\Pi^{(i)}$). *For any given PPT distinguisher $\mathfrak{D}$ that distinguishes between random variables (A, X) and (B, Y) following distributions $\mathcal{D}^A \times \mathcal{D}^X$ and $\mathcal{D}^B \times \mathcal{D}^Y$, the i -th projection distinguisher $\mathfrak{D}_\Pi^{(i)}$ is defined as follows (RAM model):*

- Allocate additional memory of size $\max\{\log(|X|), \log(|Y|)\}$ and store i .
- Replace all instructions in $\mathfrak{D}$ that access the second slot with accesses to index i .

which distinguishes between $A|_{X=i}$ and $B|_{Y=i}$.

Lemma 9. *Let X and Y be random variables having finite supports, respectively, and $\mathcal{E}$ be any subset event of X . Then for any random variables A and B , and PPT distinguisher $\mathfrak{D}$ distinguishing (A, X) and (B, Y) , the following inequality holds:*

$$\text{adv}_{\mathfrak{D}}((A, X), (B, Y)) \leq \max_{i \notin \mathcal{E}} \text{adv}_{\mathfrak{D}_{\Pi}^{(i)}}(A|_{X=i}, B|_{Y=i}) + \Pr[X \in \mathcal{E}] + 2\Delta(X, Y).$$

Proof. Take any PPT distinguisher $\mathfrak{D}$, then

$$\begin{aligned} & \text{adv}_{\mathfrak{D}}((A, X), (B, Y)) \\ &= \left| \Pr[\mathfrak{D}(u, v) = 1 | (u, v) = (A, X)] - \Pr[\mathfrak{D}(u, v) = 1 | (u, v) = (B, Y)] \right| \\ &= \left| \sum_i \left[\Pr[\mathfrak{D}(u, v) = 1 | (u, v) = (A, i)] \Pr[X = i] \right. \right. \\ & \quad \left. \left. - \Pr[\mathfrak{D}(u, v) = 1 | (u, v) = (B, i)] \Pr[Y = i] \right] \right| \\ &\leq \sum_i \left| \Pr[\mathfrak{D}(u, v) = 1 | (u, v) = (A, i)] - \Pr[\mathfrak{D}(u, v) = 1 | (u, v) = (B, i)] \right| \Pr[X = i] \\ & \quad + \sum_i \left| \Pr[X = i] - \Pr[Y = i] \right| \Pr[\mathfrak{D}(u, v) = 1 | (u, v) = (B, i)] \\ &\leq \sum_i \left| \Pr[\mathfrak{D}_{\Pi}^{(i)}(u) = 1 | u = A|_{X=i}] - \Pr[\mathfrak{D}_{\Pi}^{(i)}(u) = 1 | u = B|_{Y=i}] \right| \Pr[X = i] \\ & \quad + 2 \cdot \frac{1}{2} \sum_i \left| \Pr[X = i] - \Pr[Y = i] \right| \\ &= \sum_{i \notin \mathcal{E}} \left| \Pr[\mathfrak{D}_{\Pi}^{(i)}(u) = 1 | u = A|_{X=i}] - \Pr[\mathfrak{D}_{\Pi}^{(i)}(u) = 1 | u = B|_{Y=i}] \right| \Pr[X = i] \\ & \quad + \sum_{i \in \mathcal{E}} \left| \Pr[\mathfrak{D}_{\Pi}^{(i)}(u) = 1 | u = A|_{X=i}] - \Pr[\mathfrak{D}_{\Pi}^{(i)}(u) = 1 | u = B|_{Y=i}] \right| \Pr[X = i] \\ & \quad + 2 \cdot \frac{1}{2} \sum_i \left| \Pr[X = i] - \Pr[Y = i] \right| \\ &\leq \sum_{i \notin \mathcal{E}} \left| \Pr[\mathfrak{D}_{\Pi}^{(i)}(u) = 1 | u = A|_{X=i}] - \Pr[\mathfrak{D}_{\Pi}^{(i)}(u) = 1 | u = B|_{Y=i}] \right| \\ & \quad + \sum_{i \in \mathcal{E}} \Pr[X = i] + 2\Delta(X, Y) \\ &\leq \max_{i \notin \mathcal{E}} \text{adv}_{\mathfrak{D}_{\Pi}^{(i)}}(A|_{X=i}, B|_{Y=i}) + \Pr[X \in \mathcal{E}] + 2\Delta(X, Y). \end{aligned}$$

hence the statement is true. $\square$

Proof of Theorem 3. To prove Theorem 3, given the protocol $\Pi_{\text{SSFHE}}^{\text{Bit}}$ and an adversary $\mathcal{A}$, we construct the corresponding adversary $\mathcal{A}^*$ as follows:

- Extract the indices of corrupted parties from the state of adversary $\mathcal{A}$. Define the set of corrupted parties as **Corrupt** and the set of honest parties as **Honest** $\triangleq \mathcal{P} \setminus \text{Corrupt}$.
- Adversary $\mathcal{A}^*$ directly samples the MAC key, then emulates $\mathcal{F}_{\text{SPDZ}}$ to generate the view of dummy parties $P_1^*, \dots, P_n^*$ and the view to be forwarded to adversary $\mathcal{A}$.
- Adversary $\mathcal{A}^*$ constructs the secret key **sk**, public key **pk**, and evaluation key **ev** for each dummy party $P_1^*, \dots, P_n^*$. If a corrupted party $P_j^* \in \text{Corrupt}$ exists, $\mathcal{A}^*$ internally executes $\mathcal{A}$ and communicates with it in place of P_j^* . If **abort** occurs during emulation, $\mathcal{A}^*$ sends $\perp$ to TTP when **Output** is invoked. If no corrupted parties exist ($P_j^* \notin \text{Corrupt}$ for all j), protocol $\Pi_{\text{SSFHE}}^{\text{Bit}}$ **KeyGen** is executed identically.
- **Simulating CircuitInit**: When $\mathcal{F}_{\text{CPMPC}}$.**CircuitInit** is invoked, if **State** $\neq 0$, all currently generated view values are forwarded to $\mathcal{Z}$, and execution is terminated. Otherwise, set **State** $\leftarrow 1$, and store in the register the number of circuit inputs and the input-to-party mapping received from TTP.
- **Simulating Input**: When $\mathcal{F}_{\text{CPMPC}}$.**Input** is invoked, if **State** $\neq 1$, all currently generated view values are forwarded to $\mathcal{Z}$, and execution is terminated. Otherwise, set **State** $\leftarrow 2$, and generate the input ciphertext ct_i^* for P_i^* based on the stored input ciphertext count and dummy party connections. If a corrupted party $P_j^* \in \text{Corrupt}$ exists, $\mathcal{A}^*$ communicates with $\mathcal{A}$ instead of P_j^* , retrieves the message m_j sent by $\mathcal{Z}$ to P_j^* , and generates the corresponding ciphertext ct_j^* . If $P_i^* \in \text{Honest}$, a dummy bit $m_i = 0$ is encrypted.
- **Simulating Output**: Retrieve the output message bit m_{res} from the functionality $\mathcal{F}_{\text{CPMPC}}$.**Output**.
 - (i) If **Corrupt** $= \emptyset$, generate the ciphertext ct'_{res} by encrypting m_{res} using **pk**. Using the identity circuit $I(m) = m$, compute $\text{ct}_{\text{res}} \leftarrow \text{Eval}^F(I, \text{ev}, \text{ct}'_{\text{res}})$. Then, emulate the actual decryption protocol (either Dec^{prev} or Dec^{new}), and forward all resulting view values to $\mathcal{Z}$.
 - (ii) If **Corrupt** $\neq \emptyset$, use the previously generated ciphertexts $\text{ct}_1^*, \dots, \text{ct}_l^*$ to compute

$$\text{ct}_{\text{res}} \leftarrow \text{Eval}^F(\mathcal{C}, \text{ev}, \text{ct}_1^*, \dots, \text{ct}_l^*).$$

Then, perform direct decryption on $\text{ct}'_{\text{res}} = (\vec{a}, b)$ to obtain m'_{res} , and modify dummy honest party P_i^* 's (fake) view as follows, depending on the decryption type:

- * If Dec^{prev} is used, and $m'_{\text{res}} \neq m_{\text{res}}$, modify the (fake) view of P_i^* as:

$$[y]_i^{q_3} \Rightarrow \begin{cases} [y]_i^{q_3} \leftarrow bn^{-1} - \vec{a}^t [\vec{s}]_i^{q_3} + [E]_i^{q_3} + \lfloor q_3/2 \rfloor, & \text{if } m_{\text{res}} = 1, \\ [y]_i^{q_3} \leftarrow bn^{-1} - \vec{a}^t [\vec{s}]_i^{q_3} + [E]_i^{q_3} - \lfloor q_3/2 \rfloor, & \text{if } m_{\text{res}} = 0. \end{cases}$$

- * If Dec^{new} is used, and $m'_{\text{res}} \neq m_{\text{res}}$, modify the (fake) share of P_i^* as:

$$[m_{\text{real}}]_i^{p_{\text{res}}} \Rightarrow \begin{cases} [m_{\text{real}}]_i^{p_{\text{res}}} + 1, & \text{if } m_{\text{res}} = 1, \\ [m_{\text{real}}]_i^{p_{\text{res}}} - 1, & \text{if } m_{\text{res}} = 0. \end{cases}$$

- (iii) In both cases, adversary $\mathcal{A}^*$ modifies party P_i 's share $[m]_i$ by adding a specific β value. Since $\mathcal{A}^*$ knows the MAC keys, it updates $[m\alpha]_i$ to $[m\alpha + \beta\alpha]_i$ to ensure the MAC remains valid.
- (iv) Finally, all generated **view** values are forwarded to environment $\mathcal{Z}$.

Now, we analyze the virtual view generated by $\mathcal{A}^*$ and the actual view generated by $\mathcal{A}$. For simplicity, we assume that $\mathcal{F}_{\text{CPMPC}}.\mathbf{CircuitInit}$, $\mathcal{F}_{\text{CPMPC}}.\mathbf{Input}$, and $\mathcal{F}_{\text{CPMPC}}.\mathbf{Output}$ are each invoked exactly once in sequence.

Define CT_{in} as the set of input ciphertexts from all parties, and let ct_{res} denote the output ciphertext. Furthermore, let e be the error associated with ct_{res} , and define CT_{in}^* and ct_{res}^* as the ciphertexts generated by the adversary $\mathcal{A}^*$ following the above described process.

We first examine the structure of view when Dec^{prev} is used. Let E denote the error introduced during the execution of Dec^{prev} . If $\text{Corrupt} = \emptyset$, then adversary $\mathcal{A}$ cannot observe the final ciphertext ct_{res} and can only access the shares within Dec^{prev} . Thus, depending on whether the environment $\mathcal{Z}$ executes **EXEC** or **IDEAL**, the two possible views are given by:

$$\begin{aligned}\text{Real}^1 &\leftarrow (\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}, m_{\text{real}}, m_{\text{real}} + e + E) \\ \text{Simul}^1 &\leftarrow (\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}^*, m_{\text{res}}, m_{\text{res}} + E).\end{aligned}$$

Here, we define:

$$\text{view}_{\text{SSFHE}} \triangleq (\text{view}_{\text{open}}, \text{pk}, \text{view}_{\text{pk}}, \text{ev}, \text{view}_{\text{ev}}).$$

Note that the ideal functionality always outputs the correct value in **view**. However, in the actual execution of $\Pi_{\text{SSFHE}}^{\text{Bit}}$, due to the definition of the scheme, there exists a probability of ϵ_5 that $m_{\text{real}} \neq m_{\text{res}}$, which provides a distinguishing advantage for $\mathcal{Z}$ between the two views.

Now, if $\text{Corrupt} \neq \emptyset$, the adversary $\mathcal{A}$ can observe the entire process of computing ct_{res} from CT_{in} via the corrupted party. Thus, environment $\mathcal{Z}$ obtains the following views:

$$\begin{aligned}\text{Real}^2 &\leftarrow (\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}, \text{ct}_{\text{real}}, m_{\text{real}}, e + E) \\ \text{Simul}^2 &\leftarrow (\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}^*, \text{ct}_{\text{res}}^*, m_{\text{res}}, E).\end{aligned}$$

Additionally, if decryption is performed using Dec^{new} ,

$$\begin{aligned}\text{Real}^3 &\leftarrow (\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}, \text{ct}_{\text{res}}, m_{\text{real}}) \\ \text{Simul}^3 &\leftarrow (\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}^*, \text{ct}_{\text{res}}^*, m_{\text{res}}).\end{aligned}$$

Here, all information related to the smudging error disappears from the overall view.

Since comparing the views in $(\text{Real}^2, \text{Simul}^2)$ is significantly easier than comparing the views in either $(\text{Real}^1, \text{Simul}^1)$ or $(\text{Real}^3, \text{Simul}^3)$, the advantage bound for $(\text{Real}^2, \text{Simul}^2)$ also serves as an upper bound for the advantage in the other cases. Thus, we proceed to compute the advantage bound for $(\text{Real}^2, \text{Simul}^2)$.

Define the event $\mathcal{E}$ as:

$$\mathcal{E} \triangleq \{e + E < B\} \vee \{e + E \geq 2^\kappa\}.$$

Then, applying Lemma 7-(ii), (iii) and Lemma 9, we obtain:

$$\begin{aligned} & \text{adv}_{\mathcal{Z}}(\text{EXEC}_{\Pi_{\text{SSFHE}}^{\text{Bit}}, \mathcal{A}, \mathcal{Z}}, \text{IDEAL}_{\mathcal{F}_{\text{CPMPC}}, \mathcal{S}, \mathcal{Z}}) \\ & \leq \max_{i \in \mathcal{E}^c} \text{adv}_{\mathcal{Z}}((\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}, \text{ct}_{\text{res}}, \mathbf{m}_{\text{res}})|_{e+E=i}, (\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}^*, \text{ct}_{\text{res}}^*, \mathbf{m}_{\text{real}})|_{e+E=i}) \\ & \quad + (B-1)2^{-\kappa} + (B-1)2^{-\kappa+1}. \end{aligned} \quad (16)$$

For all events $\{e + E = i\} \in \mathcal{E}^c$, Lemma 7-(i) ensures that $e + E = i$ and e are independent. Thus, regardless of the choice of i , the distribution of the remaining samples remains unchanged, so the advantage

$$\text{adv}_{\mathcal{Z}}((\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}, \text{ct}_{\text{res}}, \mathbf{m}_{\text{res}})|_{e+E=i}, (\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}^*, \text{ct}_{\text{res}}^*, \mathbf{m}_{\text{real}})|_{e+E=i})$$

is independent of the information contained in $e + E = i$.

Next, to apply Lemma 9, define the event $\mathcal{E}'$ as:

$$\mathcal{E}' \triangleq \{\mathbf{m}_{\text{res}} \neq \mathbf{m}_{\text{real}}\}.$$

We now compute $\Delta(\mathbf{m}_{\text{res}}, \mathbf{m}_{\text{real}})$:

$$\begin{aligned} \Delta(\mathbf{m}_{\text{res}}, \mathbf{m}_{\text{real}}) &= 2^{-1} \left(|\Pr[\mathbf{m}_{\text{real}} = 1] - \Pr[\mathbf{m}_{\text{res}} = 1]| \right. \\ & \quad \left. + |\Pr[\mathbf{m}_{\text{real}} = 0] - \Pr[\mathbf{m}_{\text{res}} = 0]| \right) \\ &= 2^{-1} \left(|\Pr[\{\mathbf{m}_{\text{real}} = 1\} \cap \mathcal{E}] + \Pr[\{\mathbf{m}_{\text{real}} = 1\} \cap \mathcal{E}^c] - \Pr[\mathbf{m}_{\text{res}} = 1]| \right. \\ & \quad \left. + |\Pr[\{\mathbf{m}_{\text{real}} = 0\} \cap \mathcal{E}] + \Pr[\{\mathbf{m}_{\text{real}} = 0\} \cap \mathcal{E}^c] - \Pr[\mathbf{m}_{\text{res}} = 0]| \right) \\ &\leq 2^{-1} \left(|\Pr[\{\mathbf{m}_{\text{real}} = 1\} \cap \mathcal{E}] + \Pr[\mathbf{m}_{\text{res}} = 1] - \Pr[\mathbf{m}_{\text{res}} = 1]| \right. \\ & \quad \left. + |\Pr[\{\mathbf{m}_{\text{real}} = 0\} \cap \mathcal{E}] + \Pr[\mathbf{m}_{\text{res}} = 0] - \Pr[\mathbf{m}_{\text{res}} = 0]| \right) \\ &\leq 2^{-1}(\epsilon_{\mathcal{S}} + \epsilon_{\mathcal{S}}) = \epsilon_{\mathcal{S}}. \end{aligned} \quad (17)$$

Thus, by applying Lemma 9, we obtain:

$$\begin{aligned} & \text{adv}_{\mathcal{Z}}(\text{EXEC}_{\Pi_{\text{SSFHE}}^{\text{Bit}}, \mathcal{A}, \mathcal{Z}}, \text{IDEAL}_{\mathcal{F}_{\text{CPMPC}}, \mathcal{S}, \mathcal{Z}}) \\ & \leq \max_{i \in \mathcal{E}'^c} \text{adv}_{\mathcal{Z}}((\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}, \text{ct}_{\text{res}}), (\text{view}_{\text{SSFHE}}, \text{CT}_{\text{in}}^*, \text{ct}_{\text{res}}^*)) \\ & \quad + (B-1)2^{-\kappa} + (B-1)2^{-\kappa+1} + 2\epsilon_{\mathcal{S}} \\ & \leq \delta_{\mathcal{S}} + (B-1)2^{-\kappa+2} + 2\epsilon_{\mathcal{S}}. \end{aligned}$$

G Arithmetic black-box(ABB) model

In this chapter, we introduce the ABB model, which is the ideal functionality definition for traditional MPC.

Ideal functionality of $\mathcal{F}_{\text{ABB}}$	
Initializ:	On input (init, p^k) for all parties, the functionality activates and store the modulus p .
Input:	On input $(\text{input}, P_i, \text{varid}, x)$ from P_i and $(\text{input}, P_i, \text{varid}, ?)$ from all other parties, with varid a fresh identifier, the functionality stores (varid, x)
Add:	On command $(\text{add}, \text{varid}_1, \text{varid}_2, \text{varid}_3)$ from all parties (if varid_1 and varid_2 are present in memory and varid_3 is not), the functionality retrieves (varid_1, x) , (varid_2, y) and store $(\text{varid}_3, x + y \bmod p)$
Multiply:	On command $(\text{multiply}, \text{varid}_1, \text{varid}_2, \text{varid}_3)$ from all parties (if varid_1 and varid_2 are present in memory and varid_3 is not), the functionality retrieves (varid_1, x) , (varid_2, y) and store $(\text{varid}_3, xy \bmod p)$
Output:	On input $(\text{output}, \text{varid})$ from all honest parties (if varid is present in memory), the functionality retrieves (varid, x) , and outputs it to the environment. If the adversary inputs OK then x is output to all parties. Otherwise $\perp$ is output to all parties.

Fig. 9: The common ideal functionality for arithmetic black-box model.